

GIT / GITHUB - Premessa

Lavoravo molto spesso su documenti di testo in Microsoft Word e talvolta volevo salvare tutte le versioni storiche dello stesso documento.

E per questo rinominavo il file e aggiungevo suffissi come versione uno, versione due e così via.

Con Git, non devi preoccuparti di questo. Git se ne occupa da solo.

Memorizza tutte le versioni storiche di ogni file e puoi tornare alla versione specifica in modo abbastanza semplice e veloce.

Di solito crei più cartelle e più file e git ti consente di tracciare in modo abbastanza semplice e automatizzato tutte le versioni di ogni file nel tuo progetto.

Git è un sistema di controllo della versione distribuito.

Git ti consente di tenere traccia dei file in qualsiasi cartella chiamata repository e git conserva la cronologia di tutte le modifiche apportate ai file in questa repository.

Se vuoi utilizzare git, devi installarlo sul tuo computer locale e dopo l'installazione di git puoi iniziare a monitorare eventuali modifiche in qualsiasi cartella sul tuo computer locale.

Puoi usarlo localmente sul tuo computer.

Anche senza Internet e in locale, avrai pieno accesso all'intera cronologia di tutte le modifiche apportate al tuo repository.

Git non dipende dalla connessione Internet.

GitHub aiuta a gestire i tuoi repository git.

Git è disponibile per diversi sistemi operativi come macOS, Windows e Linux.

Per vedere se hai git installato digita da shell: **git version**.

PERSONALIZZAZIONE TERMINALE MACOSX

1. Installazione di Iterm2 - download facilissimo.
2. Apri i link: <https://ohmyz.sh/> e <https://github.com/ohmyzsh/ohmyzsh/wiki/Installing-ZSH>
3. Nel secondo link, andare nella sezione MACOSX ed eseguire brew install zsh
4. Ora dobbiamo cambiare shell (change shell) con chsh -s /usr/local/bin/zsh se ottieni errore esegui sudo sh -c "echo \$(which zsh) >> /etc/shells"

5. Ora torniamo al primo link e installiamo oh-my-zsh.
6. Eseguiamo `sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"`
7. `git clone --depth=1 https://github.com/romkatv/powerlevel10k.git ~/powerlevel10k`
8. `echo 'source ~/powerlevel10k/powerlevel10k.zsh-theme' >> ~/.zshrc`
9. `git clone --depth=1 https://github.com/romkatv/powerlevel10k.git ${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k`
10. `open ~/.zshrc`
11. `ZSH_THEME="powerlevel10k/powerlevel10k"`

A questo punto, quando effettuerò un GITINIT su una cartella, ci saranno i branch.

Occhio: <https://github.com/romkatv/powerlevel10k?tab=readme-ov-file#oh-my-zsh>

Iniziare a lavorare con GIT, digitare ***git init*** su terminale.

Quando effettuo una git init all'interno di una cartella, git crea una "Hidden folder" nascosta che contiene tutti i file di git, la cartella è visibile con il comando `ls -la`.

La "Hidden folder" si chiama `.git`.

```
Apple ~/Prova_Git on git main ls -la at 16:18:46
total 0
drwxr-xr-x@ 3 simoneremoli staff   96 28 Giu 16:18 .
drwxr-xr-x+ 98 simoneremoli staff 3136 28 Giu 16:18 ..
drwxr-xr-x@ 9 simoneremoli staff  288 28 Giu 16:18 .git
```

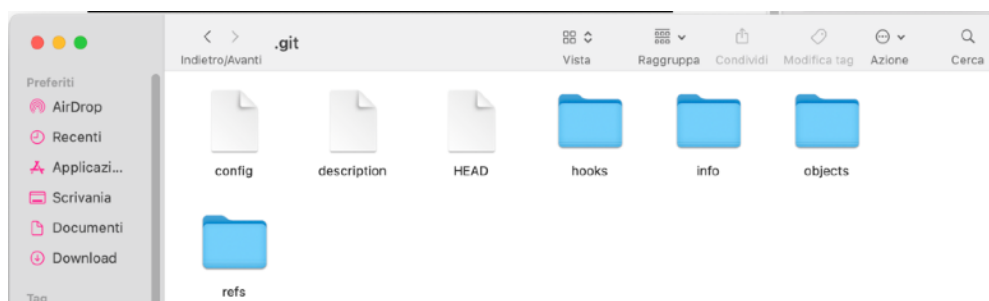
Oppure nella cartella da terminale puoi scrivere: `open .` e ti si aprirà il file system e premendo la combinazione di tasti sottostante puoi vederla dal Finder.



Ecco la cartella dot git nascosta e questa cartella è completamente gestita da git.

E per favore non apportare mai modifiche in questa cartella nascosta.

Esploriamo la cartella dentro:



Puoi vedere che, ad esempio, la cartella branch NON è presente al momento.

La cartella “objects” contiene altre due cartelle, “info” e “pack”, anch'esse vuote.



Diamo un'occhiata al contenuto di questi file, “config”, “description” e “HEAD”.

```
Apple ~/Prova_Git/.git on git main ?1 cat config ✓ at 16:37:10
[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true
  ignorecase = true
  precomposeunicode = true

Apple ~/Prova_Git/.git on git main ?1 ✓ at 16:37:13
```

È una configurazione del tuo repository git e qui vedrai alcune impostazioni predefinite.

```
Apple ~/Prova_Git/.git on git main ?1 cat description ✓ at 16:37:13
Unnamed repository; edit this file 'description' to name the repository.

Apple ~/Prova_Git/.git on git main ?1 ✓ at 16:38:02
```

E qui vedrai la descrizione del tuo repository.

```
Apple ~/Prova_Git/.git on git main ?1 cat HEAD ✓ at 16:38:02
ref: refs/heads/main

Apple ~/Prova_Git/.git on git main ?1 ✓ at 16:38:50
```

Non ti spiegherò cosa significa questa riga di codice, ma capirai perfettamente come funziona.

Nel suo file system Git memorizza gli oggetti e tutti gli oggetti sono archiviati in questa cartella "objects".
E qui al momento *non ci sono oggetti* perché abbiamo appena inizializzato il nostro nuovo **repository**. **Esistono diversi tipi di oggetti che Git può archiviare.**

Fine premessa.

TIPI DI “OGGETTI GIT”.

Blob

Tree

Commit

**Annotated
Tag**

Nei tipi di oggetti **BLOB**, Git memorizza i file.

Tutti i file con qualsiasi estensione, file video, immagini o file di testo. Tutti i file di qualsiasi tipo vengono archiviati come BLOB.

Con l'aiuto del tipo di oggetto **TREE** git memorizza le informazioni sulle directory e, come in qualsiasi altro file system, la directory può essere vuota o contenere file o contenere altre directory o avere un mix di file e directory.

TREE in git può essere un insieme di BLOB o un insieme di BLOB e altri TREE.

Ancora una volta TREE è la rappresentazione di una cartella/directory in git.

E con l'aiuto del tipo di oggetto **COMMIT**, siamo in grado di archiviare effettivamente diverse versioni del nostro progetto.

ANNOTATED TAG è in realtà un puntatore di testo persistente a un commit specifico.

E per la gestione di BLOB e TREE, utilizzeremo i cosiddetti comandi git di basso livello come "git hash-object" e "git-cat file".

Con l'aiuto del comando "git hash object", saremo in grado di creare un nuovo oggetto nella struttura git, con l'aiuto del comando "git cat file" saremo in grado di leggere oggetti git.

Ora creiamo il primo blob nel nostro nuovo progetto git.

git hash-object

git cat-file

Scrittura di un nuovo oggetto Git con git hash-object.

Abbiamo appena parlato di quattro diversi tipi di oggetti in Git e ora è il momento di creare il primo oggetto nel nostro repository Git. Creeremo **BLOB**.

Riepilogo: Quel BLOB rappresenta un singolo file nella struttura git e per la creazione di un nuovo BLOB utilizzeremo uno dei comandi di basso livello git hash-object.

Dopo la creazione dell'oggetto Git, saremo in grado di leggere le informazioni su di esso e sui suoi contenuti utilizzando il comando git cat-file.

Utilizzeremo un altro comando di basso livello "git mktree" e con questo comando saremo in grado di creare un nuovo oggetto albero.

git mktree

Qui creeremo un nuovo oggetto blob e con "git mktree" potremo creare un nuovo oggetto TREE.

COMANDO	COSA CREA
git hash-object	Crea un oggetto BLOB.
git mktree	Crea un oggetto TREE.

Ok, iniziamo creando un nuovo oggetto hash.

Ma potresti chiedermi perché questo comando ha un nome così strano.

Oggetto hash.

Perché?

Dopo verrà spiegato.

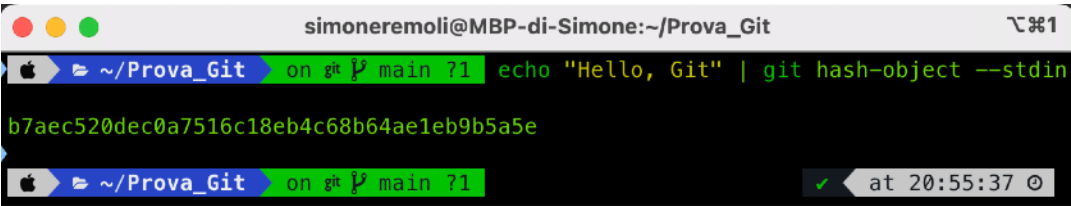
Andiamo alla Shell.

Assicurati di trovarti nella prima cartella del progetto.

Indirizzeremo l'output del primo comando (echo) a un altro comando git, vale a dire git hash object.

E sulla base di questo input, creeremo un nuovo oggetto git.

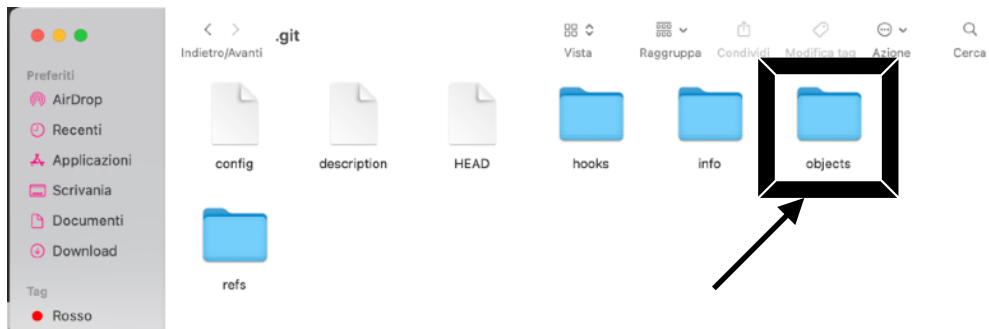
Facciamolo:



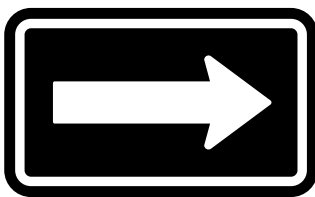
```
simoneremoli@MBP-di-Simone:~/Prova_Git
~/Prova_Git on git main ?1 echo "Hello, Git" | git hash-object --stdin
b7aec520dec0a7516c18eb4c68b64ae1eb9b5a5e
~/Prova_Git on git main ?1 at 20:55:37
```

PS. —stdin significherà che il comando git hash object prenderà il suo input dallo standard input.

Abbiamo uno strano risultato qui nel terminale, ma esaminiamo la nostra directory git e verifichiamo se lì è cambiato qualcosa.



E qui vediamo ancora le cartelle predefinite, INFO e PACK e quelle cartelle sono vuote.



Ora facciamo la seguente cosa:

```
simoneremoli@MBP-di-Simone:~/Prova_Git
~/Prova_Git on git P main ?1 echo "Hello, Git" | git hash-object --stdin -w
b7aec520dec0a7516c18eb4c68b64ae1eb9b5a5e
~/Prova_Git on git P main ?1
```

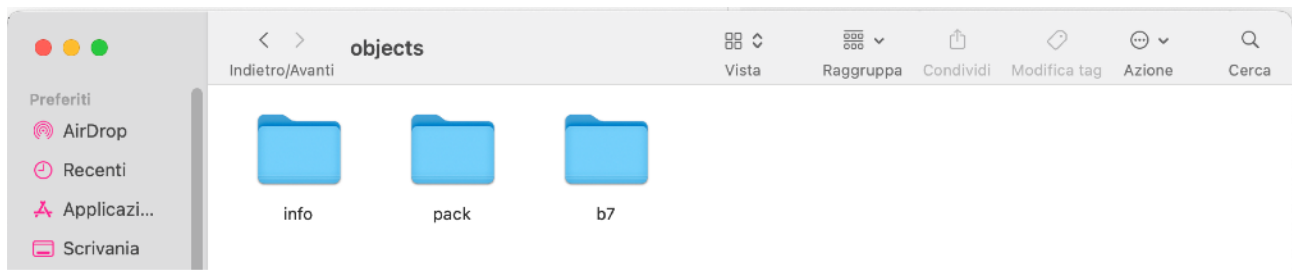
Eseguiamo lo stesso comando di prima ma aggiungiamo un'altra opzione: -w ed eseguiamo questo comando.

Stesso risultato di prima.

Ma se ora apro il contenuto della cartella, andiamo in git.

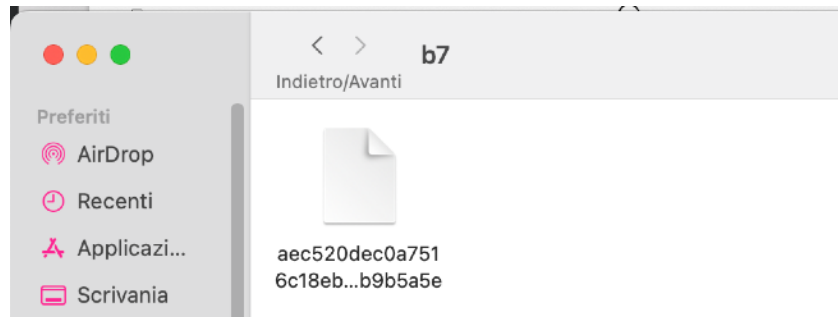
Vai agli oggetti.

Vedrò una nuova cartella chiamata B7.



E nota che le prime due lettere in questa cartella corrispondono ai primi due caratteri nella stringa B7.

Apriamo la cartella B7.



E qui dentro vedrò un file con un nome strano.

E nota che il nome di questo file termina con “5e” e nel terminale la stringa termina con “5e”.

E questo file inizia con una E e una E sono il terzo e il quarto carattere in questa stringa.

Cosa significa tutto questo? Significa che Git ha aggiunto un oggetto alla sua struttura di file.

Nel terminale vediamo questo oggetto e git ha creato per questo oggetto una nuova cartella con il nome B7.

E se concatenerai il nome della cartella B7 e il nome di questo file, otterrai questa stringa che si trova sul terminale.

E questa stringa è l'hash di questa stringa "Hello, Git".

Nome Cartella + Nome File = HASH.

JSON (JavaScript Object Notation)

E ora è il momento di rispondere alla domanda: **cos'è l'hash** e come Git memorizza gli oggetti nella sua cartella degli oggetti interni?

Ma prima di parlare di hash, lascia che ti spieghi velocemente cos'è JSON.

```
{
  "id": "1234567",
  "name": "Mike",
  "age": 25,
  "city": "New York",
  "hobbies": ["Skateboarding", "Running"]
}
```

Questo formato viene utilizzato per lo scambio di dati tra diversi server. Le parentesi graffe rappresentano il punto iniziale e il punto finale per questo oggetto.

E l'oggetto stesso è costituito da un insieme di coppie di “**CHIAVE**”-“**VALORE**”. Qui vedi chiavi come nome, ID, età, città e hobby e i valori sono diversi, prima numeri, poi stringhe oppure array.

Ma i valori in questo caso non contano.

Quello che volevo mostrarti qui è che le chiavi sono tutte distinte.

Non è possibile avere le stesse chiavi nello stesso oggetto Json.

Ogni chiave è univoca nell'intero oggetto Json.

E i valori, ovviamente, possono corrispondere.

Potresti facilmente avere gli stessi valori in diverse coppie chiave-valore.

JSON e la struttura del database git sono abbastanza simili.

Git memorizza anche chiavi e valori e la chiave in git è uguale all'hash di ciascun oggetto.

E se torno a questa cartella B7, vedrai che questa cartella, più il nome del file interno è uguale **all'hash** del file.

E, naturalmente, solo un singolo file con tale nome può essere posizionato nella cartella con il nome B7, come con Json.

Non potremmo avere due coppie chiave-valore con la stessa chiave.

Ma ti ho appena detto che in Json è possibile avere gli stessi valori in diverse coppie chiave-valore. In Git non è possibile.

Questo perché la chiave in Git viene generata in base al valore.

Ok, ora sai che il database git è in realtà un insieme di coppie chiave-valore simili agli oggetti Json.

E ora è finalmente arrivato il momento di spiegarti cos'è l'hash e come viene generato.

Quindi.

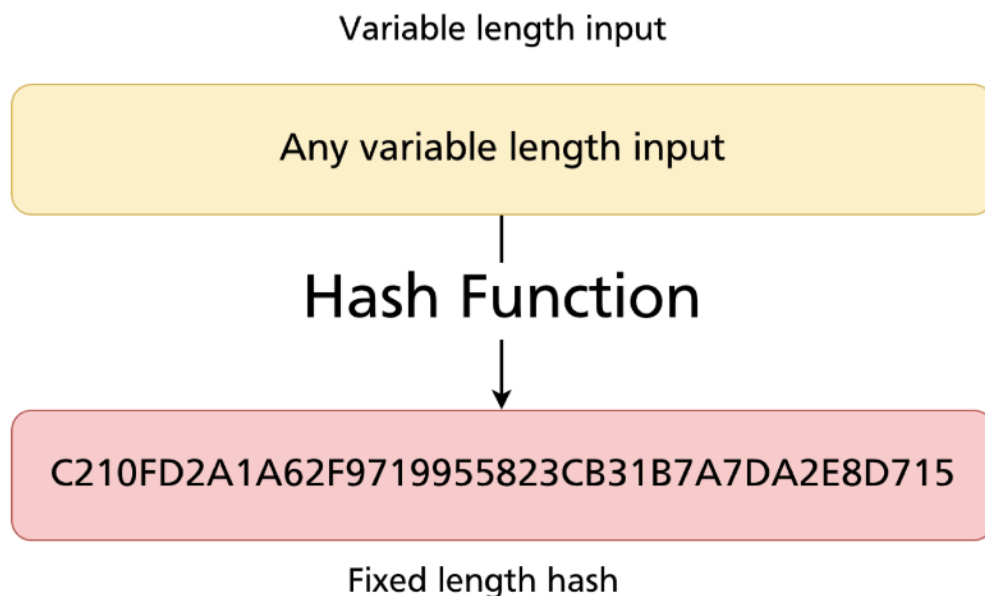
Finora abbiamo provato a utilizzare il comando git hash object con l'opzione --w per creare un oggetto git da questa stringa ("Hello, Git") e scrivere questo oggetto git nel database degli oggetti git.

E qui nel Finder abbiamo scoperto una nuova cartella nella cartella degli oggetti (.git) con il nome B7 e c'è un singolo file con questo nome all'interno di questa cartella.

Significa che nel database git memorizziamo i file in base al loro hash.

Per prima cosa creiamo l'hash dal contenuto del file (ad esempio "Hello, Git") e successivamente creiamo il file corrispondente nella cartella degli oggetti.

Ma che cos'è una funzione di HASH?



La funzione hash è una funzione che accetta qualsiasi input di lunghezza variabile e crea un hash di lunghezza fissa indipendente dall'input.

Questa lunghezza dell'hash sarà sempre fissa.

Ad esempio, possiamo prendere solo una breve stringa come abbiamo fatto qui: "Hello, Git" e generare un hash basato su questa stringa.

Ma possiamo anche prendere altri file molto più grandi di questa stringa, possiamo prendere anche, ad esempio, un file video la cui dimensione è di diversi gigabyte.

Possiamo anche scattare, ad esempio, foto o qualcos'altro.

Qualsiasi file funziona qui e la funzione hash creerà sempre un hash di lunghezza fissa.

C'è una caratteristica molto importante di ogni funzione hash.

Se conosci l'hash ma non conosci l'input, non puoi creare questo input in base all'hash.

*E tali funzioni sono chiamate funzioni unidirezionali: “**One-Way Functions**”.*

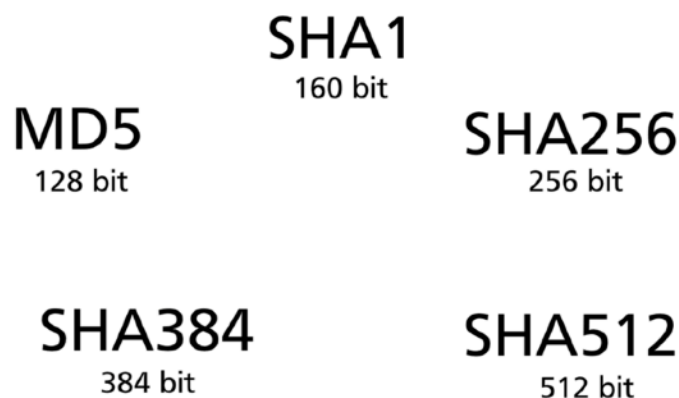
Siamo in grado di creare hash in base all'input, ma non siamo in grado di scoprire quale input è stato utilizzato per creare un hash specifico.

Ecco perché molto spesso le password che inserisci su qualsiasi sito web vengono archiviate nei database come hash.

Ricorda: lo stesso input produrrà sempre lo stesso hash;

Facciamo ora una breve panoramica di alcune funzioni hash.

Ce ne sono molte, una popolare è, ad esempio, MD5.



Questa funzione creerà un hash lungo 128 bit.

E come puoi vedere, è una dimensione piccola tra la mole di opportunità che offrono le funzioni hash.

Online esistono dei generatori delle funzioni di HASH.

SHA1

160 bits

40 hexadecimal
characters

C210FD2A1A62F9719955823CB31B7A7DA2E8D715

Hexadecimal

0 1 2 3 4 5 6 7 8 9 A B C D E F

Hex

C210FD2A1A62F9719955823CB31B7A7DA2E8D715

Base 16

Binary

1100001000010000111111010010101000011010011
0001011111001011100011001100101010101100000
10001111001011001100011011011110100111110110
100010111010001101011100010101

Base 2

Ora creiamo un hash con la funzione SHASUM:

```
simoneremoli@MBP-di-Simone:~/Prova_Git
~/Prova_Git on git main ?1 echo "Hello, Git" | shasum
c9d5d04925b93d2fb99c73ab2b5869bde7405ca4 -
~/Prova_Git on git main ?1 at 11:34:57
```

Puoi contare quei caratteri e se li conti scoprirai che ci sono esattamente 40 caratteri esadecimali.

Ora proviamo con l'opzione -n.

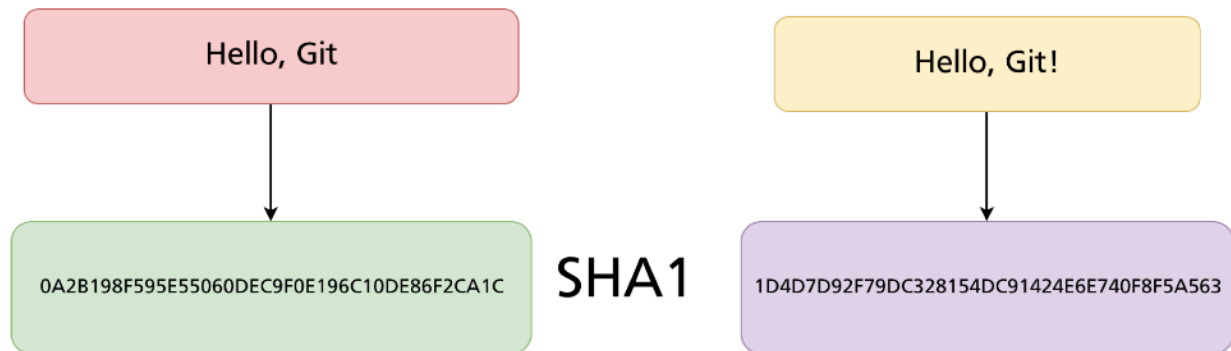
```
~/Prova_Git on git main ?1 echo -n "Hello, Git" | shasum
0a2b198f595e55060dec9f0e196c10de86f2ca1c -
~/Prova_Git on git main ?1 at 11:41:42
```

E questo secondo hash è stato generato da questa stringa senza aggiungere l'interruttore di riga.

Proviamo ora ad aggiungere il punto esclamativo alla nostra stringa.

```
simoneremoli@MBP-di-Simone:~/Prova_Git
~/Prova_Git on git P main ?1 echo -n 'Hello, Git!' | shasum
1d4d7d92f79dc328154dc91424e6e740f8f5a563 -
~/Prova_Git on git P main ?1
```

Different hashes



Ok, questi ultimi due HASH sono quelli riportati nello schema, uno generato da un input senza punto esclamativo e uno con il punto esclamativo: questi hash sono completamente differenti l'uno dall'altro.

Un altro aspetto importante è che il comando echo aggiungerà automaticamente un'interruzione di riga alla fine del testo.

E se non vuoi avere questa interruzione di riga automatica, dovresti usare il trattino con l'opzione -n.

E con questa opzione nel terminale, vedrai il segno percentuale che indica l'interruzione di riga mancante alla fine del testo.

```
simoneremoli@MBP-di-Simone:~/Prova_Git
~/Prova_Git on git P main ?1 echo -n 'Hello, Git!' | shasum
1d4d7d92f79dc328154dc91424e6e740f8f5a563 -
~/Prova_Git on git P main ?1 echo -n 'Hello, Git!'
Hello, Git!%
~/Prova_Git on git P main ?1
```

Abbiamo provato a generare hash diversi da stringhe diverse e per questo abbiamo utilizzato il comando SHASUM.

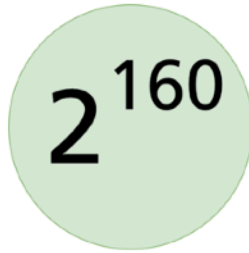
E ora, sai, un'altra caratteristica importante di ogni funzione hash.

Anche una piccola modifica nei dati di input porterà a un hash completamente diverso.

Inoltre, al momento, sai che git utilizza gli hash Sha come nomi di file e se combini il nome della cartella e il nome del file, otterrai l'hash del file.

Ricorda le combinazioni di hash:

SHA1 combinations



1461501637330902918203684832716283019655932542976

Tutto questo per rispondere alla domanda: “Quanti file Git può archiviare nello stesso repository?”.

Tanti.

Questo numero rappresenta il massimo di file che potrebbero essere archiviati in un repository git.

Ora proseguiamo con i comandi di basso livello e diamo un'occhiata al comando git cat-file.

Intanto di seguito viene mostrato il sunto del comando git hash-object.

git hash-object

```
echo "Hello, Git" | git hash-object --stdin -w
```

```
git hash-object <filename> -w
```

git cat-file options

`git cat-file -p <hash>` Contents of the object

`git cat-file -s <hash>` Size of the object

`git cat-file -t <hash>` Type of the object

Se utilizzerai l'opzione -p, il contenuto dell'oggetto specifico con l'hash Sha1 corrispondente verrà stampato sul terminale, ma ovviamente tale oggetto con l'hash specificato deve esistere nel repository git.

L'opzione successiva -s, stamperà sul terminale la dimensione dell'oggetto.

E con l'ultima opzione -t otterrai il tipo dell'oggetto.

E in precedenza abbiamo discusso del fatto che ci sono quattro tipi di oggetto in git: blob, tree, commit e annotated tag, e quando inserisci il comando git cat-file con l'opzione -t, otterrai uno di questi valori.

```
Apple ~/Prova_Git on git main ?1 echo "Hello, Git" | git hash-object --stdin
b7aec520dec0a7516c18eb4c68b64ae1eb9b5a5e
Apple ~/Prova_Git on git main ?1 git cat-file -p b7aec520dec0a7516c18eb4c68b64ae1eb9b5a5e
Hello, Git
Apple ~/Prova_Git on git main ?1 at 13:06:34
```

Otterrò "Hello, Git" qui nel terminale e questo è il contenuto del file che abbiamo archiviato nel repository git.

Ora utilizziamo l'opzione -t.

```
Apple ~/Prova_Git on git main ?1 git cat-file -t b7aec520dec0a7516c18eb4c68b64ae1eb9b5a5e
blob
Apple ~/Prova_Git on git main ?1 at 13:39:34
```

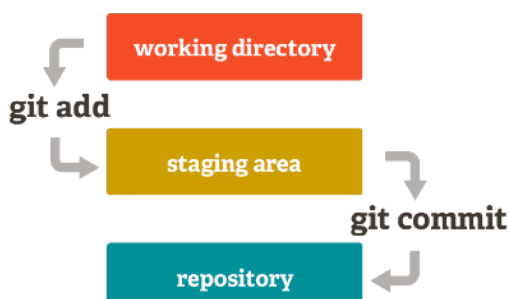
E qui vediamo che il tipo di questo oggetto con questo hash Sha1 è blob e i blob vengono utilizzati per archiviare file in git.

Ora usiamo l'opzione -s.

```
Apple ~/Prova_Git on git main ?1 git cat-file -s b7aec520dec0a7516c18eb4c68b64ae1eb9b5a5e
11
Apple ~/Prova_Git on git main ?1 at 13:46:53
```

E notiamo la dimensione in bytes: 11 bytes.

GIT DAL VIVO



Ma ora è il momento di parlare dell'**area di staging** che si trova tra la directory di lavoro e il repository git.

L'**area di staging** è solitamente chiamata indice ed è responsabile della **preparazione dei file da inserire nel repository git** e, al contrario, prepara i file presi dal repository git da mettere nella directory di lavoro.

Si noti che l'inserimento dei file nell'area di staging è un passaggio obbligatorio in tutte le operazioni git: sia quando vuoi posizionare i file dalla directory di lavoro nel repository git, sia quando vuoi leggere i file dal repository git ed estrarli nella tua directory di lavoro.

Il comando **git ls-files** viene utilizzato per elencare i file nel repository Git che sono **tracciati** (cioè gestiti da Git).

In dettaglio, il comando:

- **Mostra tutti i file tracciati** nel repository, ovvero i file che sono già stati aggiunti all'area di staging e commitati in precedenza.

Utilizzi comuni:

1. Elencare i file tracciati:

```
bash Copia codice  
  
git ls-files
```

Questo mostrerà tutti i file attualmente tracciati da Git.

2. Mostrare solo i file tracciati modificati rispetto all'ultimo commit:

```
bash Copia codice  
  
git ls-files -m
```

Questa opzione `-m` mostrerà solo i file modificati che sono già tracciati.

3. Elencare solo i file che non sono ancora stati aggiunti al repository (`git add`):

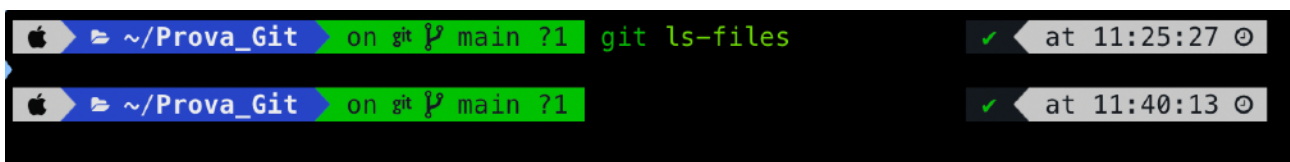
```
bash Copia codice  
  
git ls-files -o
```

L'opzione `-o` mostrerà i file "untracked", cioè quelli non ancora aggiunti.

4. Mostrare i file ignorati da `.gitignore`:

```
bash Copia codice  
  
git ls-files -i --exclude-standard
```

Questo comando mostrerà i file che sono ignorati da Git (esclusi tramite `.gitignore` o altre regole di esclusione).



Questo significa che nella **working directory** non c'è nessun file, nella **staging area** non c'è nessun file, ma ci sono dei file nella repository git.

GIT STATUS

Il comando `git status` è uno dei comandi più utilizzati in Git, ed è molto importante per verificare lo stato del tuo repository.

Esempio di output di `git status`:

```
bash Copia codice

On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   file1.txt
    new file:   file2.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)

    modified:   file3.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    file4.txt
```

Spiegazione dell'output:

- **On branch main:** Ti dice su quale branch ti trovi, in questo caso "main".
- **Changes to be committed:** Elenca i file che sono stati aggiunti all'area di staging e sono pronti per essere committati.
- **Changes not staged for commit:** Mostra i file modificati che non sono stati ancora aggiunti all'area di staging.
- **Untracked files:** Sono file presenti nella tua directory di lavoro, ma che Git non sta monitorando perché non sono stati mai aggiunti tramite `git add`.

Se vuoi creare un nuovo commit i file devono essere inseriti nell'area di staging con `git add`.

Esempio:

1. Aggiungi file all'area di staging:

```
bash Copia codice

git add file1.txt
```

2. Crea un commit con un messaggio descrittivo:

```
bash Copia codice

git commit -m "Aggiunto il file1.txt con nuove funzionalità"
```

Il comando `git commit -m` viene utilizzato per creare un **commit** in Git, associando alle modifiche un messaggio descrittivo. In questo esempio, il commit registra l'aggiunta di `file1.txt` con un messaggio che spiega le modifiche.

Se modifico un file che ho già committato, allora verrà tolto dalla zona commi.

Quando esegui un `git init`, Git imposta il branch predefinito come `master`. Tuttavia, puoi configurare Git affinché utilizzi `main` come branch predefinito invece di `master`.

Opzione 1: Cambiare il branch predefinito a livello globale

Per far sì che ogni nuovo repository che crei utilizzi `main` come branch predefinito, puoi impostare questa preferenza a livello globale.

1. Esegui il seguente comando per impostare `main` come branch predefinito globale:

```
bash
```

[Copia codice](#)

```
git config --global init.defaultBranch main
```

Questo comando cambierà il branch predefinito per tutti i nuovi repository che inizializzi con `git init`.

Uscita predefinita di `git log`:

Quando esegui `git log`, Git ti mostrerà una lista dei commit, partendo dal più recente. Ogni commit è visualizzato in questo formato:

```
bash
```

[Copia codice](#)

```
commit 1a2b3c4d5e6f7g8h9i0jklmnopqrstuvwxyz
Author: Nome Autore <email@example.com>
Date:   Sat Sep 14 10:00:00 2024 +0200
```

```
Messaggio del commit
```

- **Commit:** L'hash del commit, una stringa univoca che identifica quel commit specifico.
- **Author:** Il nome e l'email dell'autore del commit.
- **Date:** La data e l'ora del commit.
- **Messaggio del commit:** Il messaggio associato al commit, che descrive le modifiche apportate.

Opzioni comuni per `git log`:

1. **Mostrare un numero limitato di commit:** Se vuoi visualizzare solo un certo numero di commit, puoi usare il flag `-n` (dove `n` è il numero di commit da mostrare):

```
bash
```

[Copia codice](#)

```
git log -3
```

Questo mostrerà solo gli ultimi 3 commit.

2. **Visualizzare la cronologia con un'uscita più compatta:** Se preferisci vedere una versione più compatta della cronologia, puoi usare l'opzione `--oneline`, che mostra solo l'hash abbreviato e il messaggio del commit:

bash

 Copia codice

```
git log --oneline
```

L'output sarà simile a:

bash

 Copia codice

```
1a2b3c4d Messaggio del commit 1  
5f6g7h8i Messaggio del commit 2
```

3. **Visualizzare i commit con grafico:** Puoi vedere un grafico che mostra la struttura dei branch insieme ai commit usando l'opzione `--graph`:

bash

 Copia codice

```
git log --oneline --graph
```

Questo comando mostra la cronologia dei commit in modo compatto e con una rappresentazione grafica dei branch.

4. **Mostrare commit per un file specifico:** Se vuoi vedere solo i commit che hanno interessato un file specifico:

bash

 Copia codice

```
git log -- path/to/file
```

5. **Mostrare differenze tra i commit:** Se vuoi vedere il `git log` insieme alle differenze (diff) tra i commit:

bash

 Copia codice

```
git log -p
```

Questo comando mostrerà i cambiamenti effettivi (in termini di aggiunte e rimozioni di linee) tra i commit.



Il comando `git checkout` in Git ha diverse funzionalità principali, tra cui il cambio di branch, il ripristino di file, e il passaggio a un commit specifico.

Principali utilizzi di `git checkout`:

1. Cambiare branch:

Il comando più comune è quello usato per passare da un branch all'altro. Quando cambi branch, Git aggiorna i file di lavoro per riflettere lo stato del branch su cui stai passando.

```
bash
```

[Copia codice](#)

```
git checkout nome_branch
```

Questo ti porterà sul branch chiamato `nome_branch`. Se il branch non esiste localmente, Git ti dirà che non riesce a trovarlo.

2. Creare e passare a un nuovo branch:

Puoi creare un nuovo branch e passare immediatamente a esso con l'opzione `-b`.

```
bash
```

[Copia codice](#)

```
git checkout -b nuovo_branch
```

Questo comando creerà il branch `nuovo_branch` e cambierà automaticamente il branch attuale su di esso.

3. Ripristinare file a uno stato precedente:

Puoi usare `git checkout` per ripristinare un file specifico a una versione precedente (ad esempio, quella dell'ultimo commit o di un commit specifico).

- **Ripristinare un file modificato alla versione dell'ultimo commit:**

```
bash
```

[Copia codice](#)

```
git checkout -- nome_file
```

Questo comando ripristina il file `nome_file` alla versione attuale presente nel branch, annullando tutte le modifiche non ancora committate.

Utilizzi principali di `git branch`:

1. Visualizzare i branch:

Esegui semplicemente `git branch` per vedere la lista dei branch locali nel repository:

```
bash
```

[Copia codice](#)

```
git branch
```

L'output sarà una lista di branch, con un asterisco (*) che indica su quale branch ti trovi attualmente. Ad esempio:

bash

Copia codice

```
* main
  feature1
  bugfix
```

2. Creare un nuovo branch:

Puoi creare un nuovo branch con il comando:

bash

Copia codice

```
git branch nome_nuovo_branch
```

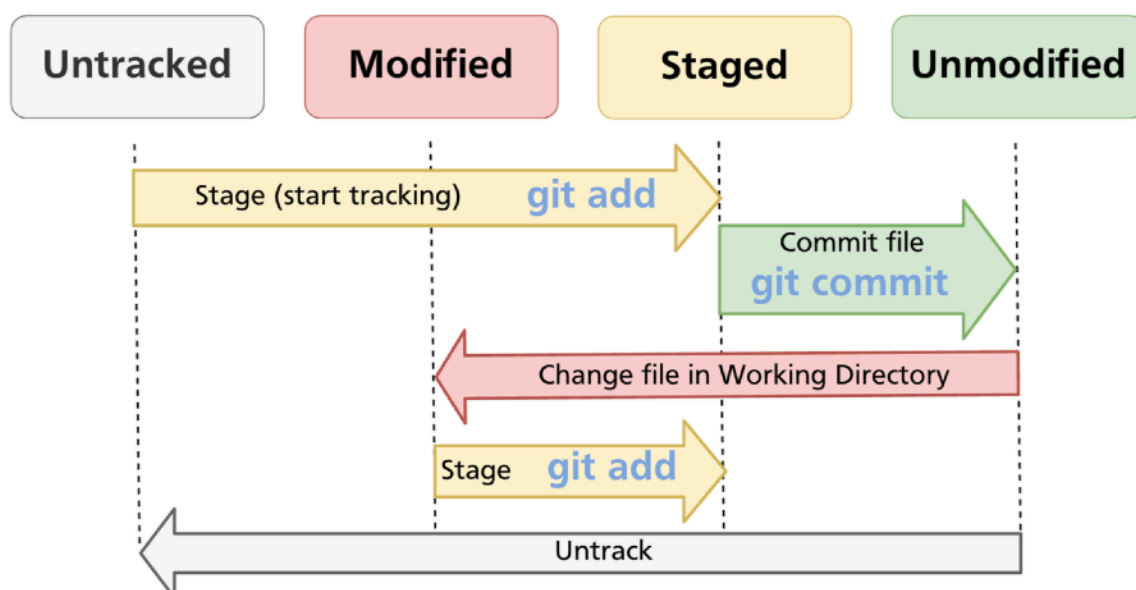
Questo comando creerà il branch `nome_nuovo_branch`, ma non passerà automaticamente a questo branch. Per passare al nuovo branch, usa il comando `git checkout nome_nuovo_branch` o `git switch nome_nuovo_branch`.

git rm --cached nomefile.

//prende il file da staging(dopo add) e lo passa in untracked. Esce dalla staging.

Quando aggiungi nuovi file nella directory di lavoro, tali file diventano “Untracked” per default, e sarai tu a decidere se tenere traccia di essi oppure no.

Git files lifecycle



Possiamo dire che il file è tracked quando è modified, staged o unmodified.

E quando si esegue il comando `git commit` e ci sono file nell'area di staging, dopo il commit tali file passeranno allo stato `unmodified`.

Il comando `git rm --cached nomefile` viene utilizzato per **rimuovere un file dall'area di staging (index)** di Git, senza eliminarlo dalla tua directory di lavoro. In altre parole, il file verrà rimosso dal controllo di Git, ma rimarrà fisicamente sul disco.

Sintassi:

bash

 Copia codice

```
git rm --cached nomefile
```

Cosa fa:

- `rm` : Rimuove un file dalla tracciatura di Git.
- `--cached` : Specifica che il file deve essere rimosso solo dall'area di staging (l'index) di Git, non dal file system.
- `nomefile` : Il file che vuoi rimuovere dal controllo di versione.

Esempio:

Supponiamo di avere un file chiamato `config.txt` che è attualmente tracciato da Git, ma vogliamo smettere di tracciare le modifiche future a quel file.

1. Rimuovi il file dall'area di staging e dal controllo di Git, mantenendo il file sul disco:

bash

 Copia codice

```
git rm --cached config.txt
```

2. Dopo aver eseguito il comando, dovrai fare un commit per confermare la rimozione del file dal controllo di Git:

bash

 Copia codice

```
git commit -m "Rimosso config.txt dal controllo di versione"
```



COMANDO PER VISUALIZZARE I FILE TRACCIATI DOPO UN GIT ADD?

Il comando che stai cercando è `git ls-files -s`. Questo comando viene utilizzato per mostrare i file tracciati nel repository Git, inclusi i dettagli dell'index, come gli hash degli oggetti, le modalità dei file, e gli stadi dei file nel caso di un merge.

Sintassi:

```
bash
git ls-files -s
```

Copia codice

Cosa visualizza:

- **Modalità del file:** Mostra i permessi dei file (come `100644` per file normali o `100755` per file eseguibili).
- **Hash dell'oggetto:** L'hash di Git (SHA-1) per il contenuto del file.
- **Stage:** Indica in quale stadio si trova il file (utile quando ci sono merge in corso).
- **Nome del file:** Il nome del file tracciato.

Esempio di output:

L'output potrebbe essere simile a questo:

```
100644 a3c5c3c5c3c5c3c5c3c5c3c5c3c5c3c5 0 file1.txt
100755 b6d6e7f8e9f8e9f8e9f8e9f8e9f8e9f8 0 script.sh
```

Copia codice

- **100644:** Permessi di lettura e scrittura per il file (`file1.txt`).
- **100755:** Permessi di esecuzione per il file (`script.sh`).
- **a3c5c3c5...:** L'hash del file `file1.txt` .
- **b6d6e7f8...:** L'hash del file `script.sh` .
- **0:** Indica che il file è nello stage 0, ossia non ci sono conflitti di merge in corso.

E nel caso in cui dovessimo modificare uno qualsiasi dei file unmodified?

Nell'output di “git status” vedrai una **sezione separata per i file modified** e l'elenco dei file modified. Nessun problema.

Quando i file sono nella sezione “unmodified” non vengono più visti con git status, ma con **git ls-files -s** sì. Ecco cosa succede quando si apportano modifiche a un file unmodified. Esso passerà allo stato modified e quando il file viene modified, puoi usare di nuovo il comando git add e spostare questo file nell'area di staging.

E infine, puoi mandare un file in "untracked" se un file specifico è già stato committato. Puoi "untrackare" questo file e spostarlo dallo stato "unmodified" allo stato “untracked”.

Ma potresti chiedermi quando dobbiamo untrackare tracciamento di uno o più file specifici?

Questo può accadere, ad esempio, quando hai aggiunto per errore un certo file o più file al tuo repository e hai già eseguito il commit di tali modifiche e vorresti rimuovere tali file dal tuo repository e renderli nuovamente non tracciati.

Inoltre, puoi anche ignorare file o cartelle specifici e, se li ignori, non li vedrai nemmeno come file non tracciati nell'output di git status.

Puoi riportare il file dallo stato di stage allo stato untracked o allo stato modified se non hai ancora eseguito il commit delle modifiche.

Se hai eseguito il commit delle modifiche e vuoi ripristinarle, dovresti usare un altro comando e ne parleremo più avanti.

BRANCH

Ad esempio, ci sono due branch, diciamo: **Master** e **new branch**.

Il branch corrente è master e, ad esempio, hai effettuato diversi commit in questo branch master.

Dopodiché sei passato al new branch e hai fatto 2 o 3 commit lì, e dopodiché torni di nuovo al Master Branch e nel Master Branch non vedrai i commit che sono effettuati fatti nel **new branch**.

Questo perché il branch corrente tiene traccia dei nuovi commit e quei commit non vengono aggiunti ad altri branch.

Ricorda: Multipli branches possono esistere all'interno della stessa repository.

Git branches management

`git branch`

List all local branches

`git branch <name>`

Create new branch

`git checkout <name>`

Checkout specific branch

`git branch -d <name>`

Delete specific branch

`git branch -m <old> <new>`

Rename specific branch

Flusso di lavoro tipico con branch:

1. Crei un branch per una nuova funzionalità o correzione:

```
bash
```

 Copia codice

```
git checkout -b nome_branch
```

2. Fai le modifiche nel branch, aggiungi i file e li committi:

```
bash
```

 Copia codice

```
git add file.txt  
git commit -m "Aggiunto file.txt"
```

3. Quando hai completato il lavoro, unisci il branch nel branch principale (ad esempio `main` o `master`):

```
bash
```

 Copia codice

```
git checkout main  
git merge nome_branch
```

4. Dopo aver unito il branch, puoi eliminare il branch che non ti serve più:

```
bash
```

 Copia codice

```
git branch -d nome_branch
```

I **branch** in Git sono fondamentali per gestire il flusso di lavoro e permettono lo sviluppo parallelo senza interferenze. Un branch è una sorta di "punto di partenza" per modifiche che consentono di lavorare su funzionalità o correzioni separate dalla linea principale di sviluppo, che di solito è il branch `main` o `master`.

Se uso: `cat .git/HEAD`

output: `ref: refs/heads/main` (puntatore di default il braccio è `main`).

Se uso: `cd .git/refs/heads/`

Output: `ls`

Output: `main temp` #presupponendo che io abbia effettuato un "git branch temp".

Se io eseguo l'eliminazione con `git branch -d git` ci permette di eliminare questo ramo **con l'opzione -d**, ma con questa opzione puoi eliminare solo i rami merged.

Ma nel nostro caso non abbiamo fatto nuovi commit all'interno di questo nuovo ramo temporaneo.

Ecco perché Git ha eliminato con successo questo ramo.

Non abbiamo perso nessuna informazione, nessun nuovo commit che è stato fatto all'interno di questo ramo.

NO MERGED BRANCH E NESSUN COMMIT DENTRO, ERGO, ELIMINATO CON `-d`.

SCOPE DEI COMMIT CON GIT LOG SU PIÙ BRANCH.

Quando crei un nuovo branch in Git, esso si basa sulla **versione attuale del branch da cui viene creato**. Quindi, se crei `branch1` mentre sei sul branch `main`, tutti i commit presenti su `main` fino a quel momento saranno visibili in `branch1`.

Esempio:

bash

 Copia codice

```
git checkout main    # Sei su 'main'
git checkout -b branch1 # Crei 'branch1' da 'main'
```

In questo scenario, `branch1` erediterà tutti i commit che erano presenti su `main` al momento della sua creazione.

Esempio:

bash

 Copia codice

```
git checkout main    # Torni su 'main'
git checkout -b branch2 # Crei 'branch2' da 'main'
```

In questo caso, `branch2` vedrà tutti i commit presenti su `main` fino a quel punto. Tuttavia, se crei `branch2` da `branch1` o da un altro branch diverso da `main`, non vedrà i commit fatti su `main` successivamente, fino a quando non unisci (merge) `main` in `branch2`.

Tuttavia, **se crei branch2 da branch1** o da un altro branch diverso da `main`, non vedrà i commit fatti su `main` successivamente, fino a quando non unisci (`merge`) `main` in `branch2`.

Esempio di flusso:

1. **Stato iniziale:**
 - `main` ha due commit (A, B).
 - Crea `branch1` partendo da `main`. `branch1` vedrà i commit A e B.
 - Fai un commit su `branch1` (C).
2. **Branch2:**
 - Torni su `main`, fai un altro commit (D).
 - Crea `branch2` partendo da `main`. `branch2` vedrà i commit A, B, e D.
3. **git log su branch1 e branch2:**
 - Se esegui `git log` su `branch1`, vedrai i commit A, B, e C.
 - Se esegui `git log` su `branch2`, vedrai i commit A, B, e D.

NOTA: I branch non vedranno automaticamente i commit fatti in altri branch dopo la loro creazione, a meno che non venga effettuato un `merge` o `rebase`.

- Se vuoi vedere la differenza tra i branch, puoi usare comandi come:

```
bash
```

[Copia codice](#)

```
git log main..branch1
```

Questo comando mostrerà i commit che sono presenti su `branch1` ma non su `main`.

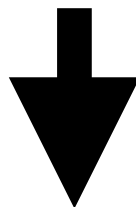
Allo stesso modo:

```
bash
```

[Copia codice](#)

```
git log branch2..main
```

Mostrerà i commit che sono presenti su `main` ma non su `branch2`.



La visualizzazione delle modifiche dei file su branch diversi:

Quando crei un nuovo branch in Git, **il nuovo branch eredita lo stato del branch da cui è stato creato**. Ciò significa che se hai fatto un commit su `main` prima di creare `branch1`, **`branch1` conterrà esattamente lo stesso stato dei file** al momento della creazione.

1. Nel branch `main`:

- Hai creato un file e hai fatto il commit.
- A questo punto, il file è tracciato e il suo stato è uguale sia nel branch `main` che in un eventuale nuovo branch che creerai, dato che Git copia lo stato corrente.

2. Creazione di `branch1`:

- Quando esegui il comando:

```
bash
```

[Copia codice](#)

```
git checkout -b branch1
```

Git creerà `branch1` basandosi sull'ultimo commit di `main`. Pertanto, **branch1** conterrà il file che hai appena committato su `main`. Non importa che tu abbia fatto il commit su `main`: una volta che crei un nuovo branch, erediterà tutto fino a quel punto.

3. Modifica successiva del file su `main`:

- Se torni su `main` e fai una modifica al file già committato, ad esempio:

```
bash
```

[Copia codice](#)

```
git checkout main
# Modifica il file
git add .
git commit -m "Modificato il file su main"
```

Questa modifica non sarà visibile in `branch1`, a meno che non faccia un merge o un rebase del branch `main` in `branch1`. A questo punto, `branch1` contiene lo stato del file come era quando è stato creato, senza le modifiche più recenti fatte su `main`.

Riassunto del funzionamento:

- Ogni branch ha il proprio spazio di lavoro e cronologia. Le modifiche fatte su un branch (come la creazione di file) **non sono visibili** sugli altri branch fino a che non si effettua un ***merge*** o un rebase.
- Il merge unisce le modifiche di un branch nell'altro, mentre il rebase riallinea la cronologia dei commit tra branch diversi.

In questo modo, Git permette di mantenere la separazione tra branch finché non si decide esplicitamente di integrare le modifiche.

Creazione di un file su `branch1`:

1. Sei su `branch1` e crei un nuovo file, ad esempio `nuovo_file.txt`:

```
bash
```

[Copia codice](#)

```
touch nuovo_file.txt
git add nuovo_file.txt
git commit -m "Aggiunto nuovo_file.txt su branch1"
```

Ora, `nuovo_file.txt` esiste solo su `branch1`, ed è stato committato lì.

2. **Passare al branch `main`**: Se torni su `main` con:

```
bash
```

[Copia codice](#)

```
git checkout main
```

Non vedrai il file `nuovo_file.txt` in `main`, perché il file è stato creato e committato solo su `branch1`.

Puoi anche vedere le differenze nei file tra i due branch con:

```
bash
```

[Copia codice](#)

```
git diff main branch1
```

Questo comando ti mostrerà le differenze tra i file modificati nei due branch.

Git è utilizzato da milioni di persone nel mondo e ci sono molti repository pubblici e puoi clonare sul tuo computer locale qualsiasi repository pubblico.

Ed è quello che faremo qui in questa sezione.

Troveremo uno dei repository pubblici, lo cloneremo sui nostri computer locali e apporteremo alcune modifiche e le committeremo.

Quando vogliamo clonare una repository in locale in realtà dobbiamo usare l'opzione `clone` e non il semplice download zip perché altrimenti non vedo la cartella `.git` nascosta. Ciò significa che se scegli l'opzione download repository e non clone, non copierai l'intero repository git e non sarai in grado di effettuare il check-out di diversi rami e diversi commit. Ecco perché dovremmo usare l'opzione `clone` invece dell'opzione `download`.

Quindi bisogna usare:

>> `git clone link_repo`

Così vedo la cartella `.git` ed è giusto così.

Cosa succede quando fai un `git clone`?

Il comando `git clone` copia l'intera cronologia del repository remoto (inclusi tutti i commit, i branch, i tag, e altre informazioni). Questa operazione crea una copia completa del repository sul tuo PC.

Cosa vedrai con `git log`?

Una volta clonato il repository, puoi usare il comando `git log` per vedere la cronologia dei commit:

```
bash
```

[Copia codice](#)

```
git log
```

Vedrai:

- **Lista dei commit:** La lista completa dei commit dal repository remoto, elencati dal più recente al più vecchio.
- **Dettagli del commit:** Per ciascun commit, vedrai informazioni come:
 - L'hash univoco del commit.
 - L'autore del commit.
 - La data del commit.
 - Il messaggio associato al commit.

Quando esegui un `git clone`, Git clona solo il branch attivo (solitamente `main` o `master`) del repository remoto, ma copia anche tutte le informazioni sui branch remoti. Tuttavia, i branch remoti non sono visibili immediatamente come branch locali.

Cosa accade con i branch dopo un `git clone`:

1. Branch attivo clonato:

- Quando fai `git clone`, il branch attivo (il branch predefinito, solitamente `main` o `master`) viene clonato e diventa il branch su cui ti trovi automaticamente nel repository locale.

2. Branch remoti copiati:

- Git clona anche le informazioni sui branch remoti, ma non li crea automaticamente come branch locali. Per vedere i branch remoti che sono stati copiati, puoi usare:

```
bash
```

[Copia codice](#)

```
git branch -r
```

Questo comando mostrerà tutti i branch remoti disponibili, ad esempio:

```
bash
```

[Copia codice](#)

```
origin/main  
origin/branch1  
origin/branch2
```

3. Branch locali: Se dopo il clone esegui il comando:

```
bash
```

[Copia codice](#)

```
git branch
```

Vedrai solo il branch attivo (ad esempio `main`), perché solo quel branch è stato clonato localmente.

Come ottenere altri branch remoti:

Se desideri lavorare su altri branch remoti, puoi **creare un branch locale che traccia un branch remoto** usando:

bash

 Copia codice

```
git checkout -b nome_branch_locale origin/nome_branch_remoto
```

Ad esempio, per creare un branch locale che traccia `origin/branch1`:

bash

 Copia codice

```
git checkout -b branch1 origin/branch1
```

Ora, puoi lavorare su `branch1` localmente, e il branch sarà sincronizzato con `origin/branch1`.

Esempio di output di `git log`:

bash

 Copia codice

```
commit e4d1b3bfa3e6b5e90e5c8f85e3f0bb5f6b16a9a3
Author: Jane Doe <jane.doe@example.com>
Date: Thu Sep 14 15:00:00 2023 +0200

    Aggiunto il supporto per il caricamento di file multipli

commit d2c5b3af3e57b5c92e09c78e79c53fb2a32a8f3a
Author: John Smith <john.smith@example.com>
Date: Wed Sep 13 10:00:00 2023 +0200

    Corretto un bug nel modulo di autenticazione
```

Filtrare i commit:

Puoi anche utilizzare varie opzioni per filtrare e personalizzare l'output di `git log`:

- Solo i commit recenti:

bash

 Copia codice

```
git log -n 5
```

Questo mostra solo gli ultimi 5 commit.

- Visualizzazione compatta:

bash

 Copia codice

```
git log --oneline
```

Questo mostra solo una riga per commit, includendo l'hash abbreviato e il messaggio.

- Visualizzare i commit di un file specifico:

bash

 Copia codice

```
git log nomefile
```

Questo mostra solo i commit che hanno modificato quel file specifico.

Se esegui una **push** da un branch locale diverso da `main` verso un repository remoto su GitHub che ha solo il branch `main`, **Git creerà un nuovo branch** sul repository remoto corrispondente al branch locale da cui stai facendo la push, a meno che non specifichi diversamente.

Cosa accade in questo scenario:

1. **Branch locale diverso da `main`**: Sei su un branch diverso da `main` nel tuo repository locale, ad esempio `feature-branch`.
2. **Push del branch locale**: Se esegui una push di questo branch con il comando:

```
bash Copia codice  
  
git push origin feature-branch
```

GitHub creerà un nuovo branch remoto chiamato `feature-branch` nel repository remoto. Il branch verrà caricato su GitHub insieme a tutte le modifiche che hai committato localmente su questo branch.

3. **Risultato su GitHub**:
 - Su GitHub, oltre a `main`, vedrai il nuovo branch `feature-branch`.
 - Il branch verrà creato automaticamente nel repository remoto con lo stesso nome del branch locale, a meno che tu non cambi il nome esplicitamente con altre opzioni di Git.

Creazione di un nuovo branch remoto:

Se vuoi controllare il comportamento della push per creare un branch con un nome diverso da quello locale, puoi usare questo comando:

```
bash Copia codice  
  
git push origin feature-branch:nuovo-branch
```

Questo comando creerà un nuovo branch remoto chiamato `nuovo-branch`, mentre localmente continuerai a lavorare su `feature-branch`.

In sintesi:

- **No**, i branch che non esistono localmente non vengono creati automaticamente quando fai un `git clone`. Solo il branch attivo del repository remoto viene clonato come branch locale.
- **Sì**, le informazioni su tutti i branch remoti vengono copiate e puoi visualizzarli e lavorarci, ma devi esplicitamente eseguire il checkout di ciascun branch remoto per creare un branch locale.

Quando scarico con git clone una repository e mostro un git status non c'è nulla da committare e l'albero è vuoto poiché i file sono tutti in unmodified. Si presenta così il setting all'atto del git clone di una repo remota. E se faccio un git log vedo tutti i commit fino a quel momento effettuati sulla repo remota (ora in locale), e posso aggiungerne di miei.

GIT DIFF

GIT DIFF: Ti mostra la differenza in due branch diversi dei file uguali che si chiamano con lo stesso nome, prima è preferibile mettere il branch creatore e dopo il figlio.

Puoi anche vedere le differenze nei file tra i due branch con:

```
bash
```

[Copia codice](#)

```
git diff main branch1
```

Questo comando ti mostrerà le differenze tra i file modificati nei due branch.

Se dal branch1 (dest) aggiungo roba nel file che non c'è nella sorgente allora vedo un +.

Se dal braccio main(sorgente) aggiungo roba che, nel file con lo stesso nome nel braccio destinazione, non c'è, allora ci sarà un -.

Se la roba è uguale non c'è nessun segno.

Usi principali di `git diff`:

1. Mostrare le differenze tra la working directory e l'ultimo commit (non ancora in staging):

Se hai modificato alcuni file ma non li hai ancora aggiunti alla staging area (`git add`), puoi vedere i cambiamenti non committati con:

```
bash
```

[Copia codice](#)

```
git diff
```

Questo comando mostra le differenze tra la tua working directory (file modificati ma non ancora aggiunti) e l'ultimo commit.

Ho mandato in commit un file ma ora lo modifico e non lo aggiungo in staging, vedo le differenze tra l'ultimo commit (con git diff) di questo file e la modifica appena effettuata.

Se rimuovo un file dalla staging area con `rm --cached file`, il file va in untracked (lo modifico) e ora faccio il commit del cambiamento, esso non apparirà più nel lavoro.

Questo è utile quando voglio prelevare un file, modificarlo e lavorarci sopra, poi quando voglio lo rendo nuovamente disponibile aggiungendolo in staging e committandolo.

PER ORA I COMMITS SONO SOLO IN LOCALE.

GIT MERGE

Posso spostare i commit da un branch figlio ad un branch padre.

1. **Crea un nuovo branch feature partendo dal branch principale (main)**
2. **Effettua modifiche nel nuovo branch e committale**
3. **Esegui il checkout del branch principale (branch di destinazione)**
4. **Unisci il branch feature nel branch di destinazione corrente**

Su main creo un file e ci scrivo qualcosa.

Aggiungo il file in staging, e mando commit.

Creo e mi sposto subito in branch1.

Branch1 eredita quel file.

Modifico il file aggiungendo roba.

Aggiungo il file in staging.

Commit.

Git checkout main.

Git merge br1, lo mergio con le modifiche fatte in branch1.

Adesso vedo le modifiche sul file sia di quelle fatte inizialmente da main e anche quelle fatte successivamente da br1.

Il **processo di merge in Git** consiste nell'unire le modifiche di due o più branch in un unico branch.

Il merge viene utilizzato quando uno sviluppatore ha lavorato su un branch separato e desidera integrare le modifiche in un altro branch (ad esempio, unire un branch secondario come `feature-branch` nel branch principale `main`).

Flusso di base del processo di merge:

1. **Inizia da due o più branch divergenti:** Ad esempio, potresti avere un branch principale chiamato `main`, e uno secondario chiamato `feature-branch`, che contiene nuove modifiche o funzionalità.
2. **Checkout del branch ricevente:** Passi al branch in cui vuoi applicare le modifiche. Ad esempio, se vuoi unire `feature-branch` in `main`, esegui:

```
bash
```

[Copia codice](#)

```
git checkout main
```

3. **Esegui il merge:** Ora esegui il merge con il branch contenente le modifiche:

```
bash
```

[Copia codice](#)

```
git merge feature-branch
```

Git tenterà di integrare tutte le modifiche da `feature-branch` in `main`.

Se su main ho un file, creo un branch figlio br1, vado su br1 che ha anche lui questo file, modifico il file, e vado su main e merge, apposto, modifiche fatte.

Ma se su main non ho un file e su br1 ho un file_x, lo modifico su br1, e dopo da main faccio merge, lo trovo integrato.

Se su main ho un file1, e su branch1 ho un file1, modifico questo file solo da branch1, lo trovo integrato.

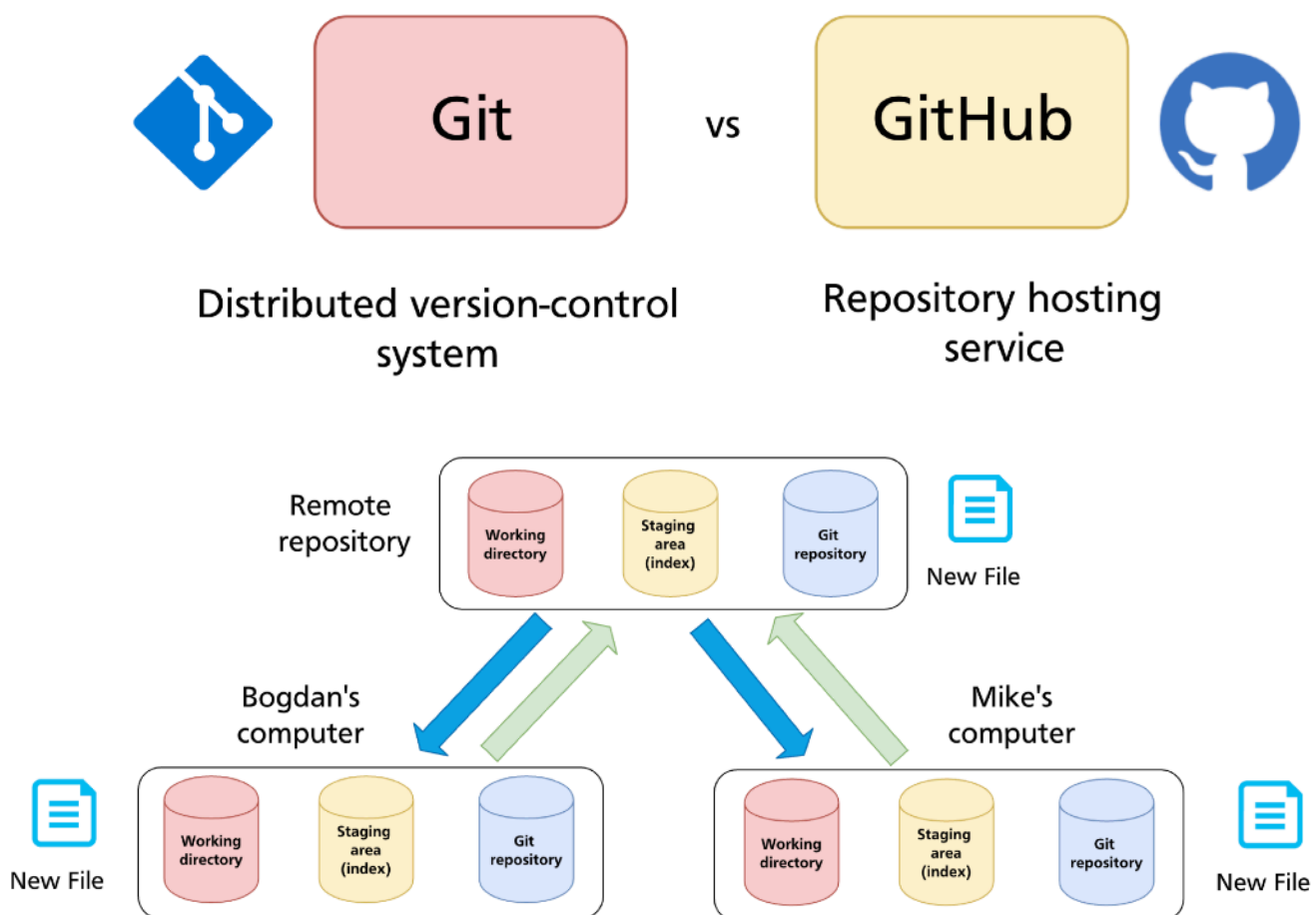
Ma se entrambi modificano un file- conflitto.

Se ho sdoppiato e dopo lo sdoppiamento il branch appena creato modifica un file e il main modifica lo stesso file dopo, se dopo da main faccio un merge ovviamente c'è un conflitto e devo modificare a mano il file problematico, e dopo devo fare l'add in staging di quel file e committare il MERGE.

Se ho sdoppiato e dopo lo sdoppiamento il branch appena creato modifica un file e il main modifica un altro file dopo, quando torno su main per unirli comunque non c'è un conflitto ma devo specificare il commit di merge. Vale anche se sdoppiato aggiunge solo un file e committa, e dopo main modifica un file diverso, e dopo sempre su main merge.

Se ho sdoppiato e il branch sdoppiato modifica un file e su main non faccio nulla, quando torno su main e faccio il merge con lo sdoppiato non devo fare nessun commit. (Non vedrò sdoppiamenti).

GITHUB E REPOSITORY REMOTE.



Nelle sezioni precedenti di questo corso abbiamo utilizzato principalmente Git e abbiamo utilizzato il repository git locale e non abbiamo ancora pubblicato il nostro repository git locale su un servizio di hosting remoto.

Se ospiti il tuo repository su GitHub e il repository è **pubblico**, significa che **chiunque nel mondo** è in grado di clonare o scaricare il tuo repository e utilizzarlo localmente sul computer.

Ovviamente puoi creare repository privati e in tal caso saranno visibili solo a te e probabilmente ai tuoi collaboratori. Ci sono anche altri servizi come **GitLab** e tutti ti consentono di ospitare i tuoi repository git nel cloud. E quando ospiti i tuoi repository git utilizzando uno dei servizi di hosting Git come **GitLab**, **Bitbucket** o **GitHub**, crei effettivamente una copia di backup del tuo repository git locale.

E se occasionalmente elimini il tuo repository git locale, sarai in grado di clonarlo di nuovo dal repository remoto e non perderai alcuna informazione.

E naturalmente servizi come GitHub ti permettono di collaborare con altre persone e lavorare sugli stessi repository e quindi Git ti permette di eseguire lo sviluppo simultaneo in diversi rami in un singolo repository. A partire dal 1° ottobre 2020, il ramo predefinito per tutti i nuovi repository creati su GitHub è "main". In precedenza era "master". Ora è "main".

Ogni volta che pushiamo un nuovo branch nel repository remoto vedremo una notifica: notifica che ci mostra che è stato appena eseguito il push di un ramo.

Vedrai tale notifica ogni volta che uno dei tuoi collaboratori eseguirà il push di rami nel tuo repository remoto.

Ora abbiamo un nostro repository remoto e, dopo averlo clonato, saremo in grado di aggiornarlo localmente. Dopodiché, pushiamo le modifiche sul nostro repository remoto.

Quando effettui un git clone Git crea automaticamente una cartella sul tuo computer con lo stesso nome del tuo repository remoto.

Dopo la clonazione saremo in grado di apportare qualsiasi modifica in questo repository localmente e dopo di che aggiornare il repository remoto usando il comando git push. Ma prima di farlo, discutiamo di come il tuo repository locale è effettivamente connesso al repository remoto e di come Git sa che c'è un repository remoto per questo repository locale.

Ok, abbiamo appena clonato il repository remoto sul nostro computer locale e ora c'è un repository remoto e un repository locale. E al momento, il nostro repository locale è connesso al repository remoto e ora possiamo interagire con il repository remoto. Nota che il repository remoto è in realtà lo stesso del repository locale in termini di struttura. C'è un'area di staging della directory di lavoro e un repository git, come sul tuo computer locale.

Cosa succede quando aggiorni il tuo repository locale? Il repository remoto si aggiorna automaticamente o no? NO! Devi **PUSHARE** le tue modifiche dal repository locale al repository remoto.

Cosa succede quando il repository remoto si aggiorna? Il tuo repository locale si aggiorna automaticamente o no? NO! Devi **scaricare** tutti gli aggiornamenti manualmente dal repository remoto.

Puoi vedere quali modifiche sono state apportate al repository remoto e dopo puoi decidere se vuoi scaricarle o meno, questo lo puoi fare tramite programmi grafici come SourceTree, GithubDesktop o VScode.

Chi ha i diritti per aggiornare il tuo repository remoto?

Ossia apportare modifiche localmente sul computer e dopo, inviare le modifiche al tuo repository remoto. Ovviamente nessuno al mondo è in grado di farlo. Devi concedere l'accesso per la scrittura al tuo repository remoto.

Chi ha i diritti per leggere il tuo repository remoto? Se il tuo repository è pubblico, chiunque al mondo è in grado di leggere dal tuo repository.

Git non ci ha chiesto alcun nome utente o password durante il git clone della repo ed è possibile solo quando il tuo repository è pubblico;

Quando è privato, solo persone selezionate possono scaricare il tuo repository remoto.

Ora tutte le comunicazioni avvengono tramite il repository remoto e quando ho apportato alcune modifiche al mio repository git, posso inviare tali modifiche al repository remoto.

E ora il mio repository e il repository remoto sono coerenti.

E ora quando un secondo programmatore vedrà che il suo repository non è coerente con il repository remoto e ci sono alcune modifiche che non sono state ancora scaricate sul suo computer, estrarrà questo repository remoto e aggiornerà il suo repository locale.

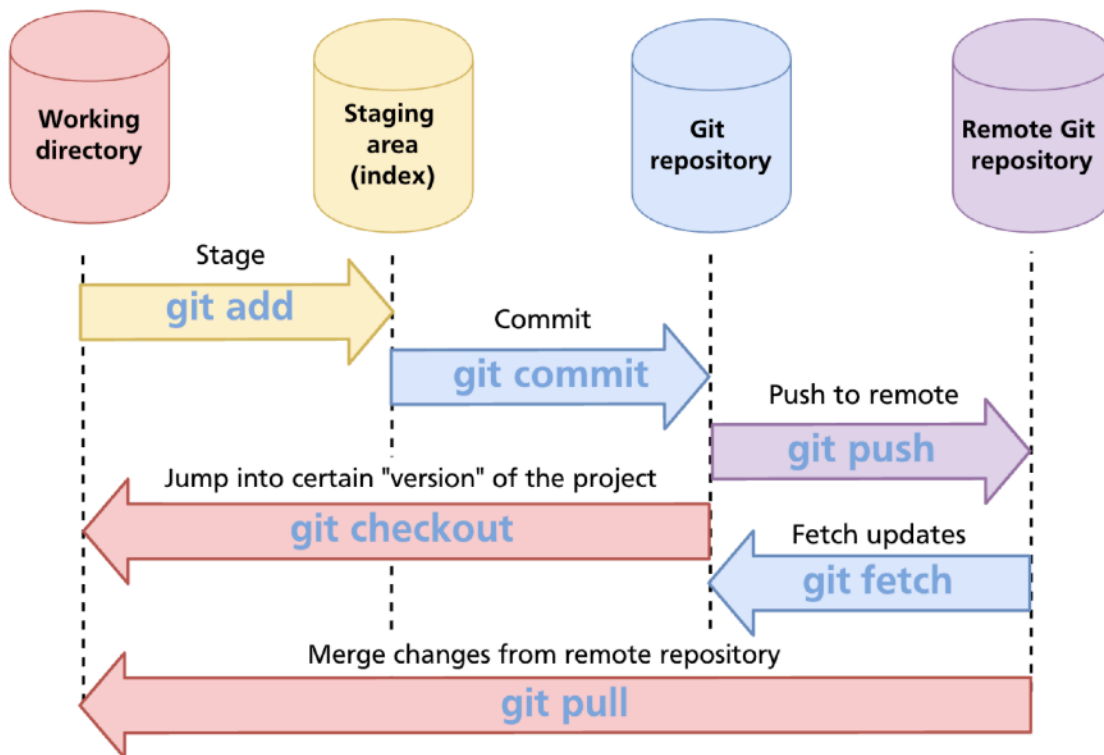
Di nuovo, ecco come avviene il processo di aggiornamento.

Ma ovviamente per questo ho bisogno di avere accesso in scrittura al repository remoto e il programmatore ha semplicemente bisogno di avere permessi di lettura per prendere quegli aggiornamenti e scaricarli sul suo computer locale.

Ma ovviamente quando il secondo programmatore vuole apportare delle modifiche e aggiornare il repository remoto, ha anche bisogno di avere permessi di scrittura.

git pull, git push e git fetch

Interaction with remote Git repository



RIPASSO GIT RM — CACHED

Ecco come funziona:

- `git rm` rimuove il file sia dalla directory di lavoro che dall'area di staging.
- `git rm --cached` rimuove il file solo dall'area di staging, ma non lo cancella dal filesystem locale.

Usi questo comando quando vuoi smettere di tracciare un file con Git (per esempio, se hai aggiunto per sbaglio un file e non vuoi che venga incluso nel commit).

Esempio:

```
bash
git rm --cached file.txt
```

In questo esempio, `file.txt` verrà rimosso dall'area di staging, ma il file sarà ancora presente nella tua directory di lavoro.

NOTA: SE FACCIO QUESTO COMANDO dopo devo fare un commit perché ho toccato l'area di staging, pertanto devo notificare che non voglio più tracciare quel file.

Solitamente si inviano al repository git remoto le modifiche che sono già state commitate al repository git locale. Per mandare in area di staging e committare in un unico comando puoi usare:

>> ***git commit -a -m "commit message"***

Naturalmente, per fare il push è necessario avere accesso in scrittura a questo repository git remoto e nessuno al mondo è in grado di effettuare il push sul tuo repository remoto.

Tieni presente, però, che è una buona norma inviare al repository remoto solo le modifiche che sono state completamente testate localmente sul tuo computer e quando sei sicuro al cento per cento che le tue modifiche non danneggeranno nulla nel progetto.

La differenza principale tra `git pull` e `git fetch` riguarda **cosa succede ai tuoi branch locali** dopo aver recuperato gli aggiornamenti da un repository remoto.

1. `git fetch`

- Il comando `git fetch` scarica tutti i dati e gli aggiornamenti dal repository remoto (come i commit e i branch aggiornati) **senza modificarne i branch locali**.
- In sostanza, aggiorna il tuo riferimento locale ai branch remoti, ma non fonde né applica automaticamente quei cambiamenti al tuo branch di lavoro attuale.
- Dopo aver fatto un `git fetch`, puoi vedere le differenze tra il tuo branch locale e quello remoto e decidere manualmente cosa fare.

2. `git pull`

- `git pull` combina `git fetch` con `git merge`. Scarica i dati aggiornati dal repository remoto e **fonde** immediatamente le modifiche nel branch corrente.
- Quindi, oltre a recuperare le modifiche come farebbe `git fetch`, esegue anche una fusione automatica delle modifiche nel branch su cui stai lavorando.
- Se ci sono conflitti tra i tuoi cambiamenti locali e quelli remoti, ti verrà chiesto di risolverli manualmente prima di completare la fusione.

Differenze in pratica:

- `git fetch` è più sicuro se vuoi solo vedere cosa è cambiato nel repository remoto senza alterare il tuo branch locale.
- `git pull` è comodo se desideri immediatamente integrare le modifiche del repository remoto nel tuo branch.
 - `origin` è un alias che Git crea per la repository remota quando cloni un repository.
 - Dopo un `git clone`, la repository locale è automaticamente associata alla repository remota tramite questo alias, che permette di sincronizzare facilmente i cambiamenti tra la copia locale e quella remota.

Ecco un esempio visivo:

```
bash
# Clonare un repository
git clone https://github.com/user/repository.git

# Mostrare i remote della repository clonata
git remote -v
# Output:
# origin https://github.com/user/repository.git (fetch)
# origin https://github.com/user/repository.git (push)
```

- Quando esegui `git pull origin main`, stai dicendo a Git di scaricare e fondere le modifiche dal branch `main` del repository remoto denominato `origin`.
- Quando esegui `git push origin main`, stai inviando i commit fatti nel branch locale `main` al repository remoto (anch'esso nel branch `main`).

Per verificare l'alias della cartella remota digitare:

```
>>git remote  
out: Origin.
```

1. Visualizzare le repository remote:

- Eseguendo `git remote -v`, Git mostra una lista delle repository remote a cui è collegata la tua repository locale, insieme agli URL associati per il fetching (scaricamento) e il pushing (inserimento) dei dati.
- Esempio:

```
bash Copia codice  
  
git remote -v  
# Output:  
# origin https://github.com/user/repository.git (fetch)  
# origin https://github.com/user/repository.git (push)
```

Ciò significa che per i comandi fetch e push verrà utilizzato lo stesso URL.

2. Aggiungere una repository remota:

- Con `git remote add <nome> <url>`, puoi aggiungere una nuova repository remota a cui collegarti. Questo è utile quando lavori con più repository remoti.
- Esempio:

```
bash Copia codice  
  
git remote add upstream https://github.com/anotheruser/repository.git
```

3. Rimuovere una repository remota:

- Con `git remote remove <nome>`, puoi rimuovere l'associazione a una repository remota.
- Esempio:

```
bash Copia codice  
  
git remote remove origin
```

4. Rinomina una repository remota:

- Puoi rinominare una repository remota usando `git remote rename <nome-attuale> <nuovo-nome>`.
- Esempio:

```
bash Copia codice
```

```
git remote rename origin upstream
```

5. Mostrare informazioni più dettagliate:

- Il comando `git remote show <nome-remoto>` fornisce informazioni dettagliate su una specifica repository remota, come i branch remoti tracciati e se ci sono cambiamenti da inviare o ricevere.
- Esempio:

```
bash
```

[Copia codice](#)

```
git remote show origin
```

Teorema: Per default, dopo la clonazione, Git non creerà rami locali corrispondenti per tutti i rami remoti, ad eccezione del ramo remoto predefinito.

Come ottenere l'elenco di tutti i rami remoti? Come vedere effettivamente tutti i rami, compresi quelli locali e remoti. Ho clonato il repository sul mio computer locale, ma se utilizzo il comando `git branch`, vedo solo il ramo master predefinito.

git branch -a

Vedrai tutti i branch locali e remoti e noterai che ogni branch remoto ha il prefisso “remote”.

Esempio di output di `git branch -a`:

```
bash
```

[Copia codice](#)

```
* main
feature1
feature2
remotes/origin/main
remotes/origin/feature1
```

In questo esempio:

- `main`, `feature1`, e `feature2` sono branch locali.
- `remotes/origin/main` e `remotes/origin/feature1` sono branch remoti, che indicano i branch della repository remota su GitHub (o altro server Git).

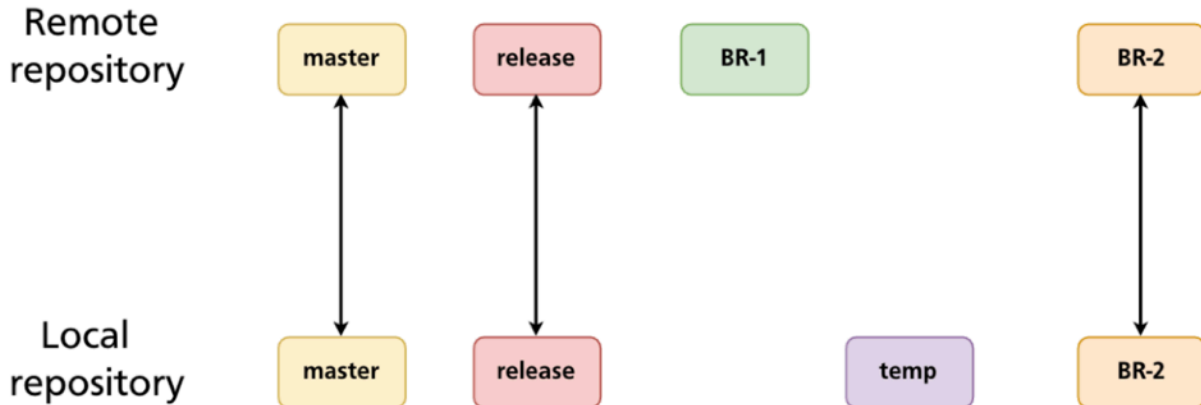
Il seguente comando viene utilizzato per elencare solo i branch remoti nella repository Git. A differenza di **`git branch -a`** che elenca sia i branch locali che remoti, questo comando mostra solo quelli remoti:

git branch -r

RAMO DI TRACCIAMENTO

Ora è il momento di spiegare cos'è il **ramo di tracciamento**. E nel nostro caso, c'è un singolo ramo di tracciamento chiamato Master e il nostro ramo master locale traccia il ramo Master remoto.

Tracking branches



Il **ramo di tracciamento** è il tuo **ramo locale che è connesso a uno specifico ramo remoto**. E per impostazione predefinita, quando cloni un repository remoto, Git crea solo un ramo di tracciamento localmente nel tuo repository git locale, per il teorema sopra.
E il nome di questo ramo corrisponde al nome predefinito del ramo nel repository git remoto.

Bene, questo ramo predefinito, nel nostro caso main, verrà creato automaticamente nel tuo repository git locale dopo il clone e questo sarà il primo ramo di tracciamento creato dopo la clonazione del repository remoto.

Nota anche che in questo diagramma c'è un ramo che è presente nel repository remoto ma è assente nel repository locale, e questo significa che non c'è un ramo di tracciamento locale per il ramo "BR1".

Potresti anche notare un altro ramo chiamato "temp" che è presente localmente nel repository locale, ma non c'è un ramo corrispondente nel repository remoto.

E questo significa che questo ramo temp non è un ramo di tracciamento.

Ovviamente puoi creare un ramo di tracciamento remoto per questo ramo "temp" creando un ramo remoto nel repository remoto.

Lo stesso per il ramo "BR-1".

Semplicemente eseguendo il check-out del ramo remoto, creerai il ramo "BR1" locale corrispondente nel tuo repository locale e questo ramo locale sarà automaticamente connesso al ramo remoto e questo ramo sarà anche un ramo di tracciamento.

Vedremo come fare.

**Ma dopo la clonazione, git ha creato solo un singolo ramo locale chiamato master.
E questo ramo è il ramo di tracciamento.**

Ora mostriamo come verificare quali branch stanno monitorando altri branch e quali no.

Il seguente comando viene utilizzato per monitorare lo stato dei tuoi branch locali e la loro relazione con i branch remoti.

git branch -vv

Il comando `git branch -vv` mostra una lista dei branch locali, indicando anche a quale commit ciascun branch è puntato e il suo stato rispetto ai branch remoti. In particolare, fornisce informazioni aggiuntive come:

1. **Commit di riferimento:** Visualizza l'ultimo commit su ciascun branch locale.
2. **Tracking dei branch remoti:** Indica a quale branch remoto (se presente) il branch locale è collegato.
3. **Stato rispetto al branch remoto:** Mostra se il branch locale è in linea con il branch remoto o se ci sono commit che lo differenziano (avanti o indietro rispetto al remoto).

Esempio di output:

bash

 Copia codice

```
main    a24dff4 [origin/main] Merge pull request #5 from feature-xyz
feature1 f2d6bff [origin/feature1: ahead 2] Work on the new feature
feature2 6d8df12 Add tests for feature2
```

Ora lascia che ti mostri come puoi creare un altro ramo di tracciamento in questo repository.

Creeremo un altro ramo di tracciamento locale per il branch remoto già esistente.

Per farlo dovremmo semplicemente usare il comando `git checkout`.

Esempio di passaggi:

1. Visualizza i rami remoti esistenti:

bash

 Copia codice

```
git branch -r
```

2. Esegui il **checkout** del ramo remoto per creare un ramo di tracking locale:

bash

 Copia codice

```
git checkout -b nome_branch_locale origin/nome_branch_remoto
```

Oppure, un comando più semplice:

bash

 Copia codice

```
git checkout nome_branch_remoto
```

Questo creerà automaticamente un ramo locale con lo stesso nome del branch remoto e lo collegherà al remoto.

Ora se eseguo il comando `"git branch -vv"` vedrò una nuova entry che identifica che il branch locale master è collegato con il branch remoto master, ed è un branch di tracciamento. Lo vedo anche con `git branch`.

```
main    8f864b9 [origin/main] Update README.md
* master 3de1761 [origin/master] compile with make
(END)
```

Ora noi abbiamo due branch locali di tracciamento sui branch remoti.

Ora supponiamo di tornare nel branch main principale ed eliminare il branch master con l'opzione:

git branch -d master

Ma ricorda che git ci consente di eliminare il ramo usando l'opzione -d solo quando il ramo è già stato unito, ma questo ramo master non è ancora stato unito né localmente né in remoto.

Ma git ci ha consentito di eliminare il ramo locale perché esiste un branch master corrispondente nel repository remoto.

Ora ricreiamo con checkout un tracciamento al branch remoto master.

(L'esempio aveva main come branch principale e master come secondario).

Ricapitolando: quando fai il clone git crea come ramo locale solo il ramo predefinito della repo e i branch remoti li vedi con git branch -r.

Per creare un tracciamento locale per un branch remoto basta fare git checkout _branch__remoto.

Ora vorrei mostrarti un altro modo per ottenere informazioni sul tracciamento dei branch.

>>git remote show origin

```
~/CL/Play on 8 & master git remote show origin
* remote origin
Fetch URL: https://github.com/SimoneRemoli/Play.git
Push URL: https://github.com/SimoneRemoli/Play.git
HEAD branch: main
Remote branches:
  main    tracked
  master  tracked
Local branches configured for 'git pull':
  main    merges with remote main
  master  merges with remote master
Local refs configured for 'git push':
  main    pushes to main    (up to date)
  master  pushes to master  (up to date)
```

```
~/CL/Play on 8 & master
```

Il comando `git remote show origin` fornisce informazioni dettagliate sul remote denominato `origin`, che è l'alias predefinito della repository remota da cui hai clonato il repository (o che hai configurato manualmente).

Ecco cosa mostra:

1. **URL del repository remoto:** L'URL del repository remoto a cui è collegato il tuo alias `origin` (può essere HTTPS o SSH).
2. **Branch tracciati:** Indica quali branch remoti sono tracciati dalla tua repository locale e fornisce informazioni su quali branch remoti corrispondono ai tuoi branch locali.
3. **Branch predefinito:** Mostra il branch predefinito del repository remoto (ad esempio, `main` o `master`).
4. **Stato del branch:** Indica se il tuo branch locale è sincronizzato con il branch remoto o se ci sono commit in sospeso (sia avanti che indietro rispetto al remoto).

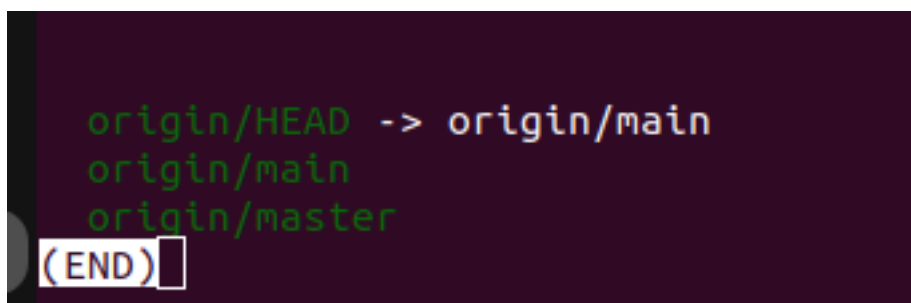
Main e master sono “tracked” perché a loro corrisponde un branch locale (di tracciamento). Sotto invece significa che ora i nostri rami possono essere utilizzati per le operazioni `git pull` e `git push`. Questo perché quei rami sono rami di tracciamento e hanno rami remoti corrispondenti.

E quando digiti "git pull master", il ramo master remoto verrà unito al ramo master locale.

Quando esegui un push, le modifiche locali dal ramo MASTER LOCALE verranno inviate al ramo MASTER REMOTO corrispondente.

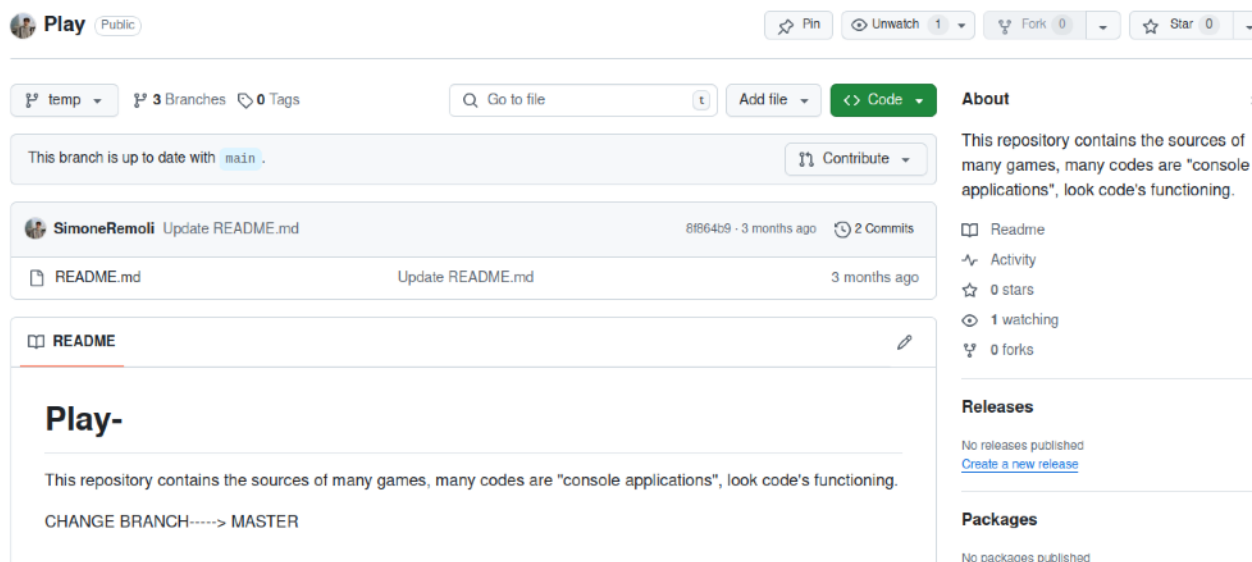
Quando apporti modifiche al tuo ramo master locale, le invierai al ramo master remoto con il comando `git push` (e qui vedrai tra parentesi che entrambi i rami MASTER e MAIN sono aggiornati (up-to-date) e ciò significa che i rami remoti e locali sono sincronizzati: non ci sono cambiamenti né nei rami remoti né nei rami locali).

Ora proveremo a creare un nuovo ramo remoto e osserveremo se vedremo questo ramo remoto nell'elenco `git branch -r`.
Ora da GitHub creo tramite interfaccia un branch “temp”: lo creo da main.
Se io effettuo un “`git branch -r`” non vedo il branch remoto temp appena creato.



```
origin/HEAD -> origin/main
origin/main
origin/master
(END)
```

Non vedo il ramo temporaneo in questa lista **ed è qui che entra in gioco il comando `git fetch`**. Con questo comando potrai recuperare modifiche remote e scaricarle nel tuo repository git locale. Ma ricorda che questa operazione non è distruttiva. Non modificherà la tua directory di lavoro o la tua area di staging. Non unirà le modifiche remote a quelle locali. Bene, ora digitiamo il comando `git fetch` e notiamo che è possibile digitare questo comando indipendentemente dal ramo in cui ci si trova al momento.



GIT FETCH IN AZIONE

```
simoneremoli@MacBook-Pro-di-Simone:~/CLONED/Play
~/CL/Play on 8 master git fetch
From https://github.com/SimoneRemoli/Play
* [new branch]      temp    -> origin/temp

~/CL/Play on 8 master
```

```
~/CL/Play on 8 master git remote -v
origin  https://github.com/SimoneRemoli/Play.git (fetch)
origin  https://github.com/SimoneRemoli/Play.git (push)
```

Dopo aver inserito il comando `git fetch`, git cerca il server remoto "origin" predefinito e accetta un URL configurato per il fetch. E puoi vedere che il nuovo ramo remoto denominato Temp è stato creato sul server remoto di "Origin" (foto grande).

Proviamo ora a dare un'altra occhiata all'elenco dei branch locali.

"git branch -vv"

Vediamo ancora solo due branch locali: MASTER e MAIN, che puntano ai rispettivi branch remoti, ma temp non c'è.

```
main    8f864b9 [origin/main] Update README.md
* master 3de1761 [origin/master] compile with make
(END)
```

Però se ora visualizzo solo i branch remoti con "git branch -r" vedo il nuovo branch "temp".

```
origin/HEAD -> origin/main
origin/main
origin/master
origin/temp
(END)
```

Ciò significa che dopo il comando git fetch git non ha creato automaticamente alcun ramo locale e dobbiamo comunque effettuare il check-out del ramo remoto se vogliamo creare il ramo di tracciamento locale corrispondente.

```
~/CL/Play on 8 master git checkout temp
branch 'temp' set up to track 'origin/temp'.
Switched to a new branch 'temp'

~/CL/Play on 8 temp
```

E solo ora è stato creato il ramo TEMP locale che tiene traccia del ramo TEMP remoto.

Ora diretto con **git branch -vv**

```
main 8f864b9 [origin/main] Update README.md
master 3de1761 [origin/master] compile with make
* temp 8f864b9 [origin/temp] Update README.md
(END)
```

E possiamo notare che il ramo locale temp appena creato è un ramo di tracciamento per la repo Origin del branch team remoto.

E ora git remote show origin e notiamo che è tutto apposto.

```
~/CL/Play on 8 temp git remote show origin
* remote origin
Fetch URL: https://github.com/SimoneRemoli/Play.git
Push URL: https://github.com/SimoneRemoli/Play.git
HEAD branch: main
Remote branches:
  main tracked
  master tracked
  temp tracked
Local branches configured for 'git pull':
  main merges with remote main
  master merges with remote master
  temp merges with remote temp
Local refs configured for 'git push':
  main pushes to main (up to date)
  master pushes to master (up to date)
  temp pushes to temp (up to date)
```

Bene, ora proviamo a eliminare questo ramo temporaneo nel repository remoto.

Andiamo graficamente su GitHub ed eliminiamo il branch remoto.

Branches New branch

Overview Yours Active Stale All

Q Search branches...

Branch	Updated	Check status	Behind Ahead	Pull request
main	3 months ago		Default	
master	6 months ago		2 14	

Ora riprovo il comando git remote show origin.

```
~/CL/Play on 7 temp git remote show origin
* remote origin
Fetch URL: https://github.com/SimoneRemoli/Play.git
Push URL: https://github.com/SimoneRemoli/Play.git
HEAD branch: main
Remote branches:
  main          tracked
  master        tracked
  refs/remotes/origin/temp stale (use 'git remote prune' to remove)
Local branches configured for 'git pull':
  main merges with remote main
  master merges with remote master
  temp merges with remote temp
Local refs configured for 'git push':
  main pushes to main (up to date)
  master pushes to master (up to date)
```

E ora vedremo in questo elenco di rami remoti che il ramo remoto temp è ora “obsoleto” = “stale” e possiamo usare il comando “git remote prune” rimuovere questo ramo dall’elenco.

```
~/CL/Play on 7 temp git remote prune origin
Pruning origin
URL: https://github.com/SimoneRemoli/Play.git
* [pruned] origin/temp
```

E il branch temp remoto è stato eliminato con successo da origin.

Se guardiamo l’output di git remote show origin ora vengono segnati solo 2 branche remoti, ossia MAIN e MASTER.

Però vedo ancora temp come branch locale - è previsto.

```
~/CL/Play on 7 temp git remote show origin
* remote origin
Fetch URL: https://github.com/SimoneRemoli/Play.git
Push URL: https://github.com/SimoneRemoli/Play.git
HEAD branch: main
Remote branches:
  main tracked
  master tracked
Local branches configured for 'git pull':
  main merges with remote main
  master merges with remote master
  temp merges with remote temp
Local refs configured for 'git push':
  main pushes to main (up to date)
  master pushes to master (up to date)
```

Pertanto se ora visualizzo i branch locali io vedo chiaramente ancora temp. Perché quando elimino un branch remoto io ho ancora quello locale, mentre se aggiungo un branch remoto devo comunque farmi un fetch e aggiungere il tracciamento locale al remoto. In questo caso ho tolto il remoto ma rimane il temp locale che precedentemente avevo messo come tracciamento.

Ora quindi visualizzo i branch locale: git branch -vv

```
main 8f864b9 [origin/main] Update README.md
master 3de1761 [origin/master] compile with make
* temp 8f864b9 [origin/temp: gone] Update README.md
(END)
```

Però se notiamo bene questo branch di tracciamento per il temp remoto (che ho eliminato) è GONE.

Ciò significa che il branch remoto TEMP sul server Origin che è monitorato dal branch locale, è stato rimosso.

Ma naturalmente *puoi creare nuovamente questo ramo remoto utilizzando*, ad esempio, il comando git push, puoi apportare modifiche a questo ramo temp locale e decidere di inviarle al server remoto, *banale, In tal caso verrà nuovamente creato un ramo remoto.*

Ma per ora rimuoviamo semplicemente questo ramo temporaneo locale.

```
~/CL/Play on temp git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

~/CL/Play on ✖ main git branch -d temp
Deleted branch temp (was 8f864b9).

~/CL/Play on ✖ main
```

Ora listiamo nuovamente i branch locali, git branch -vv:

```
* main 8f864b9 [origin/main] Update README.md
master 3de1761 [origin/master] compile with make
(END)
```

git remote show origin:

```
~/CL/Play on ✖ main git remote show origin
* remote origin
Fetch URL: https://github.com/SimoneRemoli/Play.git
Push URL: https://github.com/SimoneRemoli/Play.git
HEAD branch: main
Remote branches:
  main tracked
  master tracked
Local branches configured for 'git pull':
  main merges with remote main
  master merges with remote master
Local refs configured for 'git push':
  main pushes to main (up to date)
  master pushes to master (up to date)
```

E ora notiamo che branch locale temp non è più presente (né tantomeno il remoto che avevamo già eliminato tramite interfaccia GitHub).

Ora sai come “recuperare”/“fetch” le modifiche dal repository remoto. E ora è il momento di procedere e parlare dell'operazione **git pull**. Si noti che in quest’ultimo output non viene visualizzato l'URL di pull, ma solo l'URL di fetch e l'URL di push.

E questo perché l'operazione git pull viene eseguita parzialmente solo localmente nel tuo repository git.

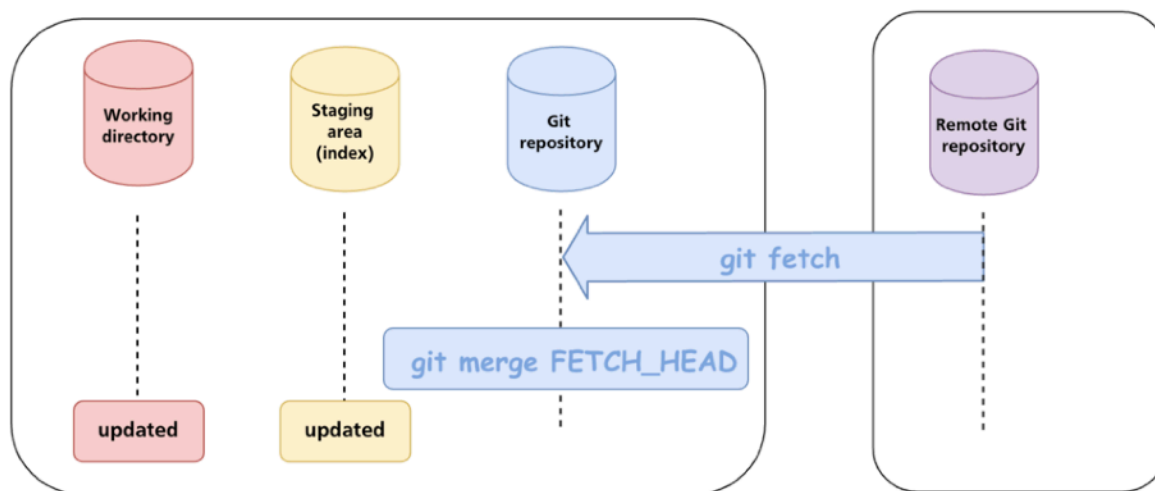
E git pull è in realtà un processo in due fasi.

Per prima cosa, recuperi le modifiche remote e dopo le unisci alle tue modifiche locali.

GIT PULL = GIT FETCH + GIT MERGE.

GIT PULL

Git pull - merges remote branch into current branch



Git pull unisce il ramo remoto nel ramo locale corrente e, naturalmente, il ramo locale corrente dovrebbe tenere traccia del ramo remoto (BRANCH DI TRACCIAMENTO).

In realtà, git pull è un processo in due fasi.

- Git esegue **git fetch** e recupera tutti gli aggiornamenti dal repository remoto e li scrive nel repository locale: ad esempio, tutti i nuovi rami, tutti i nuovi commit creati nel repository git remoto saranno creati anche nel tuo repository git locale.
- **Git merge**: ma nota che git merge viene eseguito localmente nel tuo repository git locale. Per questo non invochiamo repository remoti. E sai che nel processo di "merge" ci sono in realtà due rami e git unirà il ramo remoto al ramo locale.

Dopo git merge il ramo remoto viene unito al ramo locale e il processo di unione è esattamente lo stesso di cui abbiamo parlato prima.

E dopo l'unione, se ha avuto successo, git aggiorna ovviamente l'area di staging e la directory di lavoro e ciò avviene automaticamente perché il ramo attualmente estratto è in realtà il ramo in cui abbiamo unito il ramo remoto.

Ecco come funziona git pull.

Ma tieni presente che qui utilizziamo git merge sotto il cofano e possiamo unire solo un ramo nel ramo ricevente.

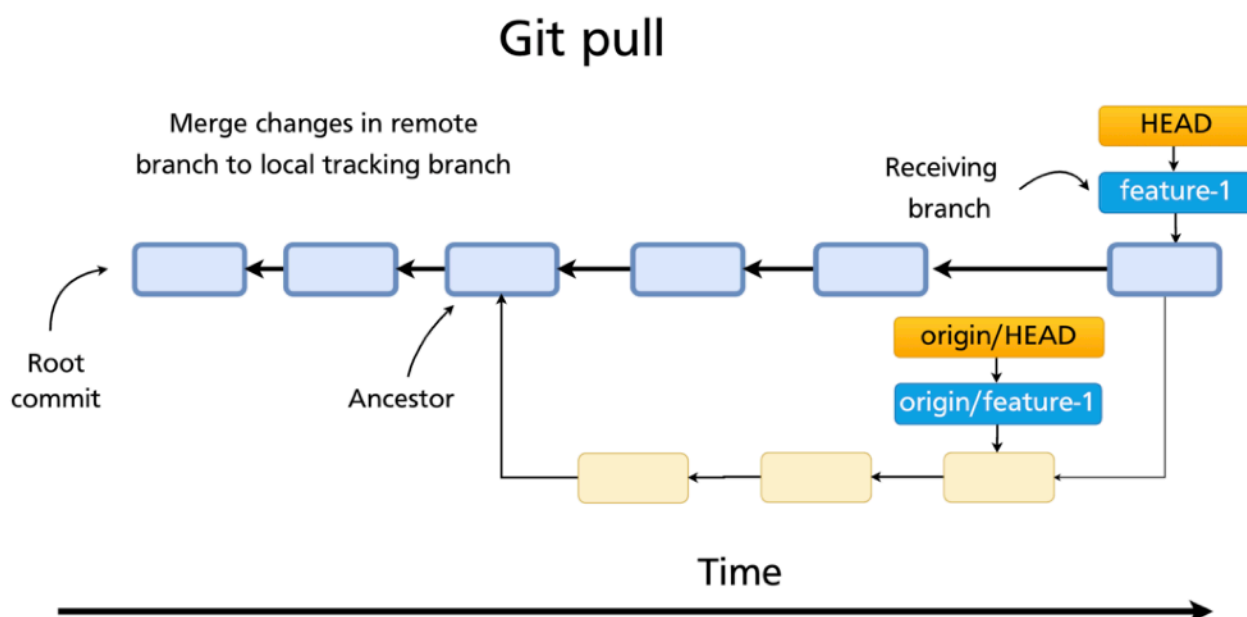
Ecco perché l'operazione **git pull aggiorna solo un singolo ramo locale che è attualmente estratto**.

Non aggiorna nessun altro ramo locale.

Solo uno che è attualmente estratto.

Tienilo a mente.

Teorema: Git pull è in realtà un processo in due fasi, prima recuperiamo le modifiche dal repository remoto e dopo uniamo automaticamente le modifiche nel branch controllato localmente. E nota ancora che dopo git pull aggiornerai solo un singolo branch controllato localmente.



1. Esegui il **checkout** del branch locale e assicurati che stia tracciando il branch remoto corrispondente. Usa `git branch -vv` per verificare.
2. Inserisci il comando `git pull`.
3. Git eseguirà il **fetch** di tutte le modifiche dalla repository remota (in background).
4. Dopo il **fetch**, Git aggiorna il file **FETCH_HEAD** che contiene gli **SHA1** degli ultimi commit nella repository remota per tutti i branch tracciati.
5. Git esegue il **merge** del branch remoto nel branch corrente con `git merge FETCH_HEAD`.

E si prega di notare ancora una volta che non c'è alcuna operazione di pull dal repository remoto: c'è solo un'operazione di fetch dal repository remoto.

Il lavoro rimanente viene eseguito solo localmente utilizzando il comando git merge.

È il branch locale che è il branch ricevitore del merge.

ORA VEDIAMO IN AZIONE, COME PRIMA COSA VEDIAMO I BRANCH LOCALI E REMOTI: `git branch -a`.

```
* main
master
remotes/origin/HEAD -> origin/main
remotes/origin/main
remotes/origin/master
(END)
```

Perfetto, abbiamo i due branch remoti e i due branch locali di tracciamento a quelli remoti. Se faccio `git branch -vv`:

```
* main    8f864b9 [origin/main] Update README.md
master 3de1761 [origin/master] compile with make
(END)
```

Ho la conferma di quanto detto.

Posso vedere la conferma ulteriore con "`git remote show origin`", ma è superfluo.

Perfetto, ora digitiamo il comando "`git fetch -v`" (l'opzione verbose).

```
~/CL/Play on 8 0 main git fetch -v
POST git-upload-pack (202 bytes)
From https://github.com/SimoneRemoli/Play
 = [up to date]      main      -> origin/main
 = [up to date]      master    -> origin/master

~/CL/Play on 8 0 main
```

Git ha recuperato gli aggiornamenti da questo URL, ma non c'è nulla da aggiornare localmente perché entrambi i rami MASTER e MAIN sono aggiornati e puoi vedere queste informazioni qui.

Ora è tutto sincronizzato sul main e sul master, quindi se ad esempio da master effettuo un **GIT PULL**, è tutto sincronizzato. Ecco l'output:

```
~/CL/Play on 8 0 main git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

~/CL/Play on 8 0 master git pull
Already up to date.
```

UTILIZZIAMO GIT PULL CON L'OPZIONE VERBOSE:

```
~/CL/Play on 8 0 master git pull -v
POST git-upload-pack (204 bytes)
From https://github.com/SimoneRemoli/Play
 = [up to date]      master    -> origin/master
 = [up to date]      main      -> origin/main
Already up to date.
```

ATTENZIONE: Il blocco di output di "git pull -v" è uguale al blocco di output di "git fetch -v".

Ecco perché ti ho detto che git pull esegue prima git fetch.

E dopo, se ci sono altre modifiche nel ramo remoto, proverà a unirle al tuo ramo locale.

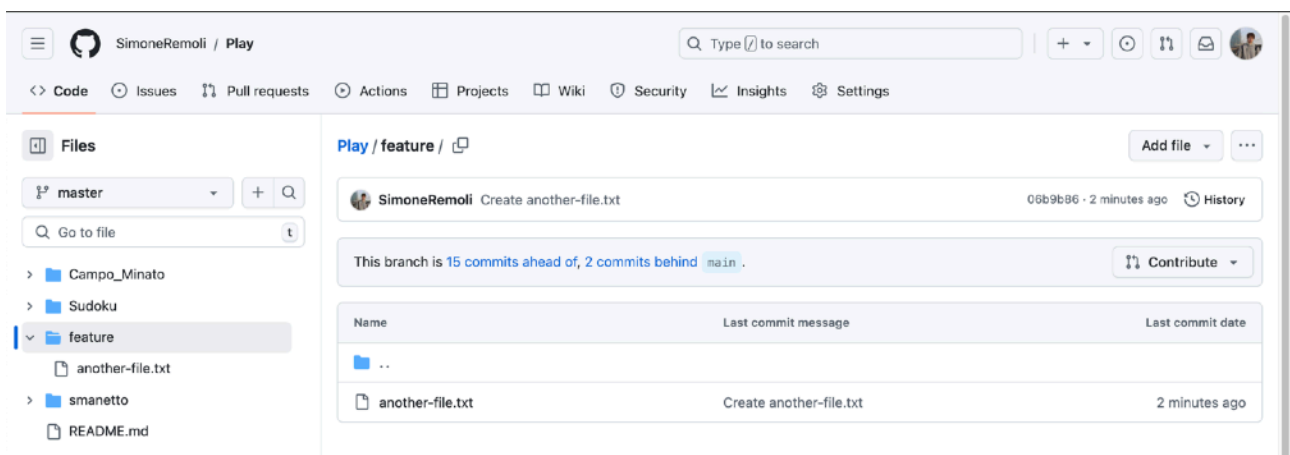
Ok, ora switchiamo nel branch principale main: "git checkout main".

Se facciamo lo stesso procedimento con "git pull -v" e "git fetch -v" vedremo gli stessi risultati.

Quindi git pull non produrrà alcun risultato.

E ora apportiamo effettivamente delle modifiche al ramo remoto e proviamo a eseguire l'operazione **git pull**.

Ora andiamo al nostro repository remoto e apportiamo delle modifiche. Andiamo nel branch MASTER e creiamo un nuovo file chiamato "another-file.txt" dentro una nuova cartella denominata "feature".



Assicurati di essere ora nel ramo MASTER nel terminale. È il ramo ricevente e ora le modifiche dal ramo remoto verranno unite al nostro ramo ricevente.

Ora finalmente digitiamo il comando **"git pull -v"** con l'azione verbose per vedere anche gli aggiornamenti del comando:

```
~/CL/Play on 8 master git pull -v
POST git-upload-pack (204 bytes)
POST git-upload-pack (924 bytes)
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 1004 bytes | 167.00 KiB/s, done.
From https://github.com/SimoneRemoli/Play
   3de1761..06b9b86  master    -> origin/master
   = [up to date]    main      -> origin/main
Updating 3de1761..06b9b86
Fast-forward
 feature/another-file.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 feature/another-file.txt

~/CL/Play on 8 master
```

Ora è curioso come, all'interno del branch locale master che traccia il branch remoto in origin master, io possa eseguire il comando sotto e vedere che git pull abbia già effettuato la fusione del nuovo file dentro il branch locale.

```
~/CL/Play on 8 master git ls-files -s
100644 c655a09b6280f9b3255e645f3d0f8744b895e465 0 Campo_Minato/campominato.c
100644 0a808fc17a99091d92db72a12826ca4f394568cc 0 README.md
100644 dbb81230c23e81ad440dcd0cdcc83db9e15e7b69 0 Sudoku/Makefile.txt
100644 b6edd287c5e4c19822cce1f1289ae54bf025367 0 Sudoku/main.c
100644 e4857f4f0e2af20ac0b58a1d3fb08e030180418a 0 Sudoku/problems.h
100644 b9659aa8ff359021dc05e4bd6597c5acea4c36a2 0 Sudoku/solver.c
100644 e5f4da3afb6fdb1331a94bf5765a22dda5df5b0 0 Sudoku/validator.c
100644 5676ce8d39663d3d836eec978f36c74de10c7d27 0 feature/another-file.txt
100644 0b98b957c7d065cfff0d4bb5557d198967f7750e 0 smanetto/Tris/tris.c
100644 c8ecfba7af2d5d01a43e78b7d261194407b7d219 0 smanetto/impiccato/config.txt
100755 eb0255990d70ee0b671cf6cbc615f1a6c79b6ac1 0 smanetto/impiccato/impi
100644 8ff3def92b9379534ef2273a65235db385a07f4e 0 smanetto/impiccato/impiccato.c
```

Dall'output del pull puoi notare che il ramo MAIN è aggiornato.

Ma ci sono stati dei cambiamenti nel ramo MASTER e il ramo MASTER prima puntava a questo commit 3DE1761, ma ora punta a questo commit 06B9B86, ossia l'ultimo commit della creazione del file nuovo.



Il branch LOCALE master ancora punta al commit precedente 3DE1761 ma quello remoto punta al commit 06B9B86.

E ORA L'AGGIORNAMENTO DEL BRANCH LOCALE AVVIENE SOTTO NELLA SEZIONE UPDATING BEN VISIBILE.

In questo caso è possibile eseguire un "fast forward update" perché il nostro ramo MASTER non è stato modificato dopo questo commit.

Non abbiamo eseguito alcuna modifica e non abbiamo eseguito alcun commit nel ramo MASTER, come per il merge normale, se sdoppio e modifico un file nel ramo sdoppiato, quando torno nel ramo principale e faccio il merge non devo specificare nulla.

Se listo in master vedrò la cartella feature con dentro il file nuovo:

```
~/CL/Play on 8 master ls
Campo_Minato README.md Sudoku feature smanetto
```

Se ora mostro un git log mi accorgo che l'ultimo commit coincide con l'ultimo commit del branch master remoto. In sostanza se abbiamo un ramo locale di tracciamento al ramo remoto, se a questo'ultimo effettuo delle modifiche, basta usare git pull per vederle aggiornate ed integrarle in locale.

Bene, ora eseguiamo la stessa operazione, ma faremo alcuni nuovi commit sia nel ramo remoto MASTER che nel ramo locale MASTER e vedremo cosa succede.

Ora ho creato da interfaccia grafica un file chiamato "list.txt" dentro la cartella feature nella repo remota e da terminale ho effettuato il classico git pull -v e me la sono trovata aggiornata nel branch di tracciamento locale. Quindi dentro feature ho un file che si chiama "another-file.txt" e "list.txt".

```

~/CL/Play/feature on ✖ master git pull -v ✓< with simoneremoli@MBP-di-Simone
POST git-upload-pack (204 bytes)
POST git-upload-pack (974 bytes)
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 1004 bytes | 167.00 KiB/s, done.
From https://github.com/SimoneRemoli/Play
   06b9b86..c932a76  master    -> origin/master
   = [up to date]    main      -> origin/main
Updating 06b9b86..c932a76
Fast-forward
 feature/list.txt | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 feature/list.txt

~/CL/Play/feature on ✖ master ls ✓< with simoneremoli@MBP-di-Simone
another-file.txt list.txt

```

Ora andiamo a modificare il file another-file.txt!

```

~/CL/Play/feature on ✖ master nano another-file.txt

~/CL/Play/feature on ✖ master !1 git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
       modified:   another-file.txt

no changes added to commit (use "git add" and/or "git commit -a")

~/CL/Play/feature on ✖ master !1 git add another-file.txt

~/CL/Play/feature on ✖ master +1 git commit -m "modificato un file another.txt localmente master"
[master b009412] modificato un file another.txt localmente master
1 file changed, 3 insertions(+)

~/CL/Play/feature on ✖ master !1 ✓< with simoneremoli@MBP-di-Simone at 15:09:31

```

Ora qui il funzionamento è il medesimo per le repo locali. Una volta modificato il file locale al branch devi nuovamente portare in staging dalla sezione “modified” e committare il messaggio. Ed è quello che ora è stato fatto.

Ora da interfaccia GitHub dentro la cartella feature ho creato il file: “one-more-file.txt”.

Play / feature /
Add file
...

SimoneRemoli Create one-more-file.txt ac65553 · 1 minute ago History

This branch is 17 commits ahead of, 2 commits behind main . Contribute

Name	Last commit message	Last commit date
..		
another-file.txt	Create another-file.txt	3 hours ago
list.txt	Create list.txt	23 minutes ago
one-more-file.txt	Create one-more-file.txt	1 minute ago

Ora ci sono cambiamenti sia nel repository remoto che in quello locale:

1. Ho modificato il locale il file “another-file.txt”;
2. Ho aggiunto un file in remoto “one-more-file.txt”.

Ora vediamo l'ultimo commit eseguito nella repository locale nel braccio master di tracciamento, corrisponde alla modifica del file "another-file.txt".

```

git log
commit b00941293debbcad0837388abacf29879b6d07ba (HEAD -> master)
Author: Simone Remoli <simoneremoli00@gmail.com>
Date: Sat Sep 21 15:09:31 2024 +0200

    modificato un file another.txt localmente master

commit c932a76556e1ff12c34b7432d84bf8bf345f5993 (origin/master)
Author: Simone Remoli <simoneremoli00@gmail.com>
Date: Sat Sep 21 15:02:48 2024 +0200

    Create list.txt

commit 06b9b86b99fd19ee99bc607ce2024bb0e5edaa42
Author: Simone Remoli <simoneremoli00@gmail.com>
Date: Sat Sep 21 11:59:57 2024 +0200

    Create another-file.txt

commit 3de1761ecd3df7c5e74a52be8fd7d8f35443add8
Author: Simone Remoli <simoneremoli00@gmail.com>
Date: Fri Mar 22 18:56:22 2024 +0100

    compile with make

```

Questo commit è ASSENTE nella repository remota. Gli ultimi due sono commit che ho eseguito nel branch remoto su interfacci grafica GitHub e dopo li ho portati nel branch con un git pull, il primo non è ancora stato portato in locale:

Create one-more-file.txt	Verified	ac65553			
SimoneRemoli committed 7 minutes ago					
Create list.txt	Verified	c932a76			
SimoneRemoli committed 29 minutes ago					
Create another-file.txt	Verified	06b9b86			
SimoneRemoli committed 3 hours ago					

Ora provo ad eseguire il fetch e dopo applico il merge. E ci verrà chiesto di aggiungere testo per il merge commit perchè ci sono cambiamenti sia nel ramo ricevente locale che nel remoto. Stiamo eseguendo il merge nel ramo ricevente.

```
➤ ~/CL/Play/feature on 8 master ↓1 ↑1 ls
another-file.txt list.txt

➤ ~/CL/Play/feature on 8 master ↓1 ↑1 git fetch

➤ ~/CL/Play/feature on 8 master ↓1 ↑1 git merge FETCH_HEAD
Merge made by the 'ort' strategy.
feature/one-more-file.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 feature/one-more-file.txt

➤ ~/CL/Play/feature on 8 master ↑2
```

[illegible]

```
git log
commit bf611ba058c2b3168413db61f621770712cb6e40 (HEAD -> master)
Merge: b009412 ac65553
Author: Simone Remoli <simoneremoli00@gmail.com>
Date: Sat Sep 21 16:05:39 2024 +0200

Merge branch 'master' of https://github.com/SimoneRemoli/Play

commit ac6555327289bc3e91b5acec252b199c008db39b (origin/master)
Author: Simone Remoli <simoneremoli00@gmail.com>
Date: Sat Sep 21 15:24:54 2024 +0200

Create one-more-file.txt

commit b00941293debbcad0837388abacf29879b6d07ba
Author: Simone Remoli <simoneremoli00@gmail.com>
Date: Sat Sep 21 15:09:31 2024 +0200

modificato un file another.txt localmente master

commit c932a76556e1ff12c34b7432d84bf8bf345f5993
Author: Simone Remoli <simoneremoli00@gmail.com>
Date: Sat Sep 21 15:02:48 2024 +0200

:
```

Così dentro il file di tracciamento ho aggiornato i commit, ossia il commit fanno in remoto della creazione di “one-more-file.txt” e il commit del branch.
In fin dei conti ho modificato un file diverso dal file “one-more-file.txt” remoto da mergiare, perché in locale ho modificato il file “another-file.txt”, e quindi questo è un commit nel branch dove devo effettuare l’unione(merge), pertanto devo specificare il commit.

*NOTA: NON SAREBBE CAMBIATO NULLA SE AL POSTO DI AGGIUNGERE E POPOLARE UN FILE IN REMOTO COME “ONE-MORE-FILE.TXT” AVESSI MODIFICATO LIST.TXT IN REMOTO, GIÀ PRESENTE ANCHE IN LOCALE, PERCHÈ TANTO IL BRANCH RICEVENTE HA MODIFICATO “ANOTHER-FILE.TXT”.
SE IL BRANCH RICEVENTE LOCALE MODIFICAVA “LIST.TXT” AVEVO UN CONFLITTO.*

Git ha unito con successo le modifiche dal ramo MASTER remoto al ramo MASTER locale.

E ora puliamo il nostro terminale e proviamo ad effettuare di nuovo il PULL per estrarre gli aggiornamenti.

Ci possiamo aspettare che ora è tutto sincronizzato tra i branch remoti e i branch locali.

```
~/CL/Play/feature on ✖ master git pull ✓ with simoneremoli@MBP-di-Simone at 16:26:39
Already up to date.

~/CL/Play/feature on ✖ master ✓ with simoneremoli@MBP-di-Simone at 17:35:56
```

Ora, ricordi che avevamo aggiornato il file in locale “Another-file.txt”? Vediamo il suo contenuto in locale:

Nella repo remota non vedo questo aggiornamento:

```
nano another-file.txt
File: another-file.txt
UW PICO 5.09

un nuovo file creato nel branch master

modificato il file localmente
```

Play / feature / another-file.txt

SimoneRemoli Create another-file.txt

Code Blame 1 lines (1 loc) · 39 Bytes

1 un nuovo file creato nel branch master

Questo perché al momento tali modifiche si trovano solo NEL BRANCH MASTER locale: Non vengono incorporate nel repository remoto, ma tutte le modifiche apportate dal repository remoto vengono ora incorporate nel nostro repository locale.

Così funziona git pull quando ci sono cambiamenti nel repository remoto e nel repository locale.

Ora è il momento di andare avanti.

E poi proveremo a cambiare gli stessi file negli stessi posti, sia nel repository remoto che nel repository locale.

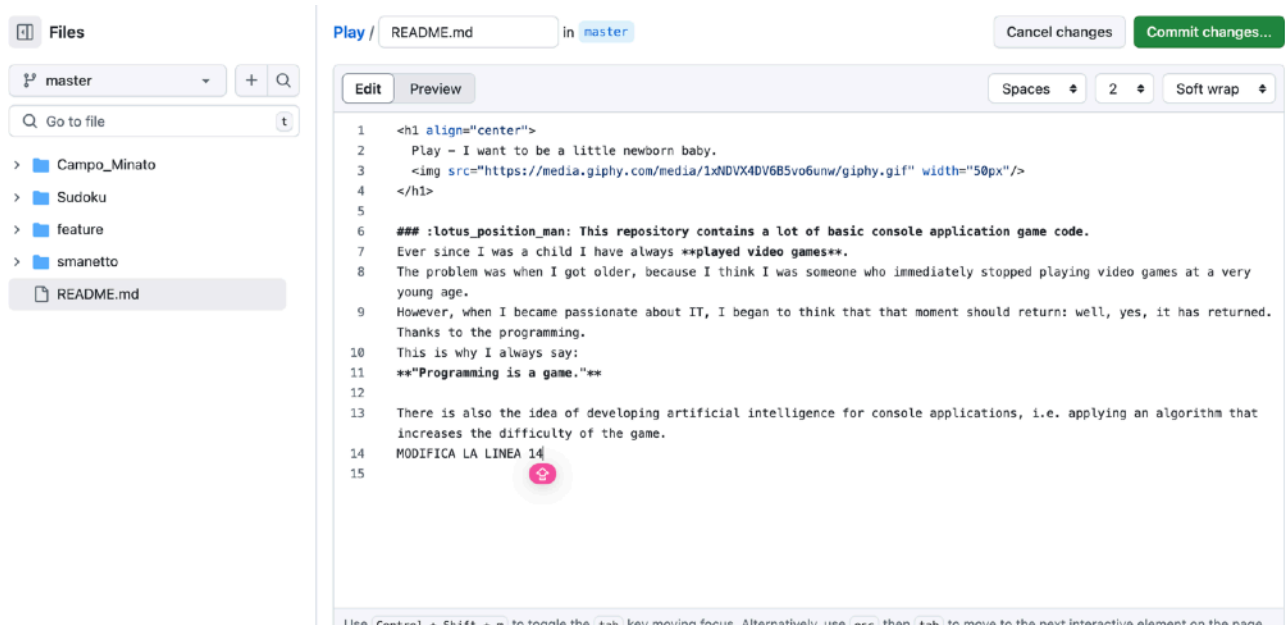
E puoi immaginare cosa succederà.

Ci sarà un conflitto di merge e vedremo come saremo in grado di risolvere tale conflitto.

Proveremo a creare un conflitto di unione durante l'operazione git pull e per questo faremo delle modifiche nel MASTER remoto e nel ramo MASTER locale nello stesso file nello stesso posto.

E dopo proveremo a eseguire l'operazione git pull.

Andiamo su GitHub, assicuriamoci di essere nel ramo MASTER e modifichiamo il file Readme.



Torniamo ora al terminale e modifichiamo anche qui il file Readme.md nel branch locale.



Ovviamente ora il file è stato modificato e pertanto verrà aggiunto nuovamente in staging e committato.

```
simoneremoli@MBP-di-Simone:~/CLONED/Play
~/CL/Play on 8 master ↵ !1 git status ✓< with simoneremoli@MBP-di-Simone at 18:38:27
On branch master
Your branch is ahead of 'origin/master' by 2 commits.
(use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

~/CL/Play on 8 master ↵ !1 git add README.md

~/CL/Play on 8 master ↵ +1 git commit -m "Modificato localmente README.MD"
[master a025abb] Modificato localmente README.MD
1 file changed, 1 insertion(+), 1 deletion(-)

~/CL/Play on 8 master ↵ 1 ✓< with simoneremoli@MBP-di-Simone at 18:38:42
```

Ovviamente questa modifica locale sarà assente ora nella repository remota e viceversa la modifica della repository remota non sarà presente nella locale. E ora proviamo a eseguire l'operazione git pull. E ancora, notiamo che abbiamo modificato lo stesso file negli stessi punti nel repository remoto e nel repository locale.

```
simoneremoli@MBP-di-Simone:~/CLONED/Play
~/CL/Play on 8 master ↵ 1 128 X with simoneremoli@MBP-di-Simone at 19:12:32
~/CL/Play on 8 master ↵ 1 128 X with simoneremoli@MBP-di-Simone at 19:13:05
git fetch
git merge FETCH_HEAD
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

IL MERGE È FALLITO.

```
simoneremoli@MBP-di-Simone:~/CLONED/Play
<h1 align="center">
  Play - I want to be a little newborn baby.
  
</h1>

### :lotus_position_man: This repository contains a lot of basic console application game code.
Ever since I was a child I have always **played video games**.
The problem was when I got older, because I think I was someone who immediately stopped playing video games at a v
young age.
However, when I became passionate about IT, I began to think that that moment should return: well, yes, it has ret
d. Thanks to the programming.
This is why I always say:
***"Programming is a game."**

<<<<<<< HEAD
There is also the idea of developing artificial intelligence for console applications, i.e. applying an algorithm
increases the difficulty of the game. Modifica locale 14
=====
There is also the idea of developing artificial intelligence for console applications, i.e. applying an algorithm
increases the difficulty of the game.
MODIFICA LA LINEA 14
>>>>>>> 8331b571ef9953050365a71949acdb87de0c2eb7
```

Bisogna risolvere manualmente modificando il file README.md!

Scrivi per inserire testo

